

More from Less: 3D Gaussian Splatting Digital Twins with Iterative Expansion and Trajectory Compensation for Sparse-Demonstration Bimanual Manipulation

Anonymous Authors

Abstract

Sparse bimanual demonstrations cover limited object states and contact configurations. More from Less combines structured action prediction, failure-region expansion, and local recovery. A vision-language-action (VLA) policy predicts joint action chunks with shared context and training-only geometric supervision. Object-level 3D Gaussian Splatting supplies appearance assets for physical proxies with separate collision geometry and assigned physics. Policy failures locate regions for virtual rollouts. Only complete trajectories passing task, physics, and safety checks enter real-dominated retraining. During deployment, asynchronous video references provide image-plane targets for current-state mapping and gated fusion. Base and fused actions pass through a shared quadratic program and nonlinear recheck. Across six real-robot task categories, Full Pipeline reaches a four-stage macro-average success rate of 89.00%, compared with 75.83% for Structured Dual-arm VLA. This comparison measures stage-level outcomes of the combined system, rather than complete-task success.

Project page: <https://robovla.github.io/More-from-Less/>.

Keywords: Bimanual manipulation, Sparse demonstrations, Vision-language-action, 3D Gaussian Splatting, Failure-region expansion, Trajectory compensation

1. Introduction

Bimanual manipulation requires the arms to coordinate around shared objects while maintaining contact and avoiding collisions. A small demonstration set may cover only a narrow range of object states and contact configurations. Few-shot imitation and cross-embodiment learning transfer skills from limited demonstrations [1–4], but execution can fail outside those trajectories. Gaussian Splatting reconstruction [5, 6] provides scene representations, and generative video models [7–11] predict candidate visual futures. Using these resources for recovery requires both a way to locate policy failures and a way to reject infeasible actions.

This problem concerns both the base policy and its training distribution. Even when a bimanual vision–language–action (VLA) policy shares observations and jointly predicts both arms’ actions, relative poses, contact states, and step-specific roles may be learned only implicitly through action errors, making stable coordination difficult under sparse demonstrations. Visual data augmentation [12, 13], simulation benchmarks, and domain randomization [14, 15] can improve data efficiency and broaden training distributions. However, random perturbations or uniform sampling do not actively focus on the current policy’s failure regions, and trajectories admitted without constraint-based screening may introduce behaviors that cannot be executed.

High-fidelity reconstruction provides a visual foundation for targeted expansion. Gaussian Splatting methods, including 3D Gaussian Splatting (3DGS) and its geometric refinements [5, 6], preserve multiview appearance and geometric cues, while feature-assisted object-level neural fields [16–25] further support instance separation and editing. These visual representations do not inherently contain collision geometry, movable joints, or contact parameters, however, and therefore cannot directly serve as the physical proxies needed for policy rollouts.

Even after the training distribution is expanded, real-world execution remains affected by perception noise, contact slip, occlusion, and unseen intermediate states. A deviation at a local step can interrupt an entire bimanual task. Repeated execution of the original policy neither supplies new local targets nor ensures that recovery actions satisfy bimanual coupling and collision constraints. A deployment-time recovery mechanism must therefore cooperate with the base policy and allow unreliable references to be rejected.

Generative video models [7–11] can propose candidate futures conditioned on current observations and language, and model-based object pose estimation

[26–32] can provide geometric alignment cues. Neither is equivalent to a robot command. Executable recovery additionally requires bimanual action semantics, reachability, and collision constraints to be established, with the base policy retained or fallback invoked when a reference is unreliable.

To address these gaps, we propose the More from Less closed-loop framework, illustrated in Fig. 1. The system first learns a structured bimanual VLA using shared task context, joint action outputs, and loss-level relation supervision. It then combines object-level 3DGS appearance, conservative geometry, structural priors, and calibrated physical parameters to construct physical proxies in Isaac Sim. Virtual trajectories are generated around failure regions and accepted through explicit checks, after which the updated policy is rolled out in the proxies again. During deployment, low-frequency video references are requested asynchronously only after a step transition, a risk-threshold crossing, or a detected failure. References contribute to action fusion only after alignment with the current state and validity checks. All executed actions ultimately pass through quadratic programming (QP) safety projection.

The main contributions are as follows.

(1) A structured bimanual VLA interface combines shared task context, joint action prediction, and training-only relation supervision. The action-dependent relations specified in this paper are relative end-effector pose and inter-arm distance. Contact and role annotations provide training context; the results assess the structured configuration as a whole, without establishing separate contact- or role-loss effects.

(2) Object-level 3DGS physical proxies support sampling near policy failures. Appearance assets are combined with collision geometry, articulated structure, and assigned physical parameters. Candidate rollouts enter training only after task, physics, and safety checks. Repeating failure discovery after policy updates defines the expansion procedure.

(3) Step-level compensation uses risk and reference validity to gate an asynchronous video reference. Current-state alignment and local action mapping convert its image trajectory into joint targets. Both base and fused actions pass through a shared QP and nonlinear recheck; rejected commands trigger slow or hold behavior.

In held-out real-robot evaluation comprising six bimanual task categories, the complete system increases the macro-average success rate over four task-stage categories from 75.83% for the structured bimanual VLA to 89.00%. Comparative experiments, ablations, and stress tests further examine the

three technical modules and their applicability under the tasks and evaluation protocol considered here.

2. Related work

2.1. *Sparse demonstrations, video imitation, and bimanual VLA policy learning*

Recent video imitation methods learn manipulation by matching demonstration image sequences to execution observations or by recovering object motion [1–4]. Such direct correspondence can transfer tasks when viewpoints, objects, and task stages are closely aligned, but remains sensitive to appearance and camera changes. Point tracking [33], dense correspondence, semantic matching, and imagination-based world models [34–36] subsequently introduced intermediate geometric or affordance representations, preserving structural information in demonstrations while reducing reliance on pixel-level consistency. A recent RAS review likewise treats demonstration collection, representation learning, and policy optimization as coupled processes rather than independent stages [37].

To reduce dependence on paired robot demonstrations, subsequent work learns transferable priors from large collections of unpaired videos [36, 38–40]. These methods typically predict affordances, contact regions, object motion, or interaction directions rather than directly regressing robot commands [38, 41–47], improving data scalability and transfer across instances. Nevertheless, embodiment differences, viewpoint changes, and task-stage misalignment still prevent visually plausible interactions from being translated directly into executable bimanual action sequences.

Bimanual imitation introduces additional dependencies: the arms are coupled through shared objects, contact ordering, and relative end-effector geometry. Prior work shows that imitation objectives can adapt task-relevant motion features to single-arm and bimanual constraints [48]. However, when modern VLA backbones are trained mainly using separate action errors for the left and right arms, relative poses, contact stages, and leader–follower roles in sparse demonstrations remain only implicitly supervised. Increasing backbone capacity alone does not supply this relational signal. Recent RAS research on dynamic bimanual manipulation further shows that learned task parameters must remain coupled to coordinated motion under feasibility constraints [49]. Bimanual trajectory planning also jointly addresses collision-free action ordering and coordination [50]. Action Chunking with Transformers (ACT)

provides a direct baseline for bimanual action chunking [51], while OpenVLA demonstrates how a pretrained VLA backbone can be adapted as a replaceable module for downstream manipulation [52].

Our structured interface adds geometric relation supervision to joint bimanual action prediction. Relative end-effector pose and minimum inter-arm distance are computed from the candidate target configuration. Contact and role annotations enter as fixed training context, with their role in the loss specified in Section 3.1.3. The experiments compare the resulting Dual-arm VLA configuration with ACT, Diffusion Policy, and other baselines; they do not test the same relation loss across all backbones.

2.2. Object-level 3D scene representations, physical proxies, and data expansion

NeRF-based 3D segmentation [53], neural implicit surfaces [54], and 3D Gaussian Splatting [5, 6] have established foundations for high-quality multi-view reconstruction and real-time rendering. Dynamic representations, 4D scene models, and generative 3D methods extend this scope to changing scenes and asset generation [55–60]. These developments improve visual fidelity and scene coverage, but a rendering representation does not automatically supply the collision geometry, inertial parameters, friction, or articulated-structure constraints required for manipulation-policy rollouts.

Object-level representations based on semantic features, feature distillation, and neural-field decomposition [16–25] facilitate instance localization, editing, and tracking. Object-NeRF [61], Clip-NeRF [62], and SPIn-NeRF [63] further demonstrate controllable object editing. Compared with scene-level reconstruction, these methods improve semantic separation. However, occluded back surfaces, merged boundaries, open surfaces, and coordinate systems tied to the source scene can still make reconstructed objects unsuitable for repeated-contact simulation.

Visual augmentation [12, 13], semantic editing [64], simulation benchmarks, and domain randomization [14, 15] broaden training distributions, while diffusion-based or world-model policy rollouts [65–67] increase candidate-state diversity. These approaches are effective when useful states remain close to the existing distribution. Uniform perturbation, however, does not prioritize rare contact failures, articulated-joint boundaries, or hazardous bimanual configurations. Scene-graph task planning, object-centric motion fields, and digital-twin resources [68–70] add structural information, yet do not inherently connect real-policy failures, physical predicates, and screened data feedback.

Research on articulated-object manipulation shows that perception, structural representations, and action timing must remain coupled when object configurations change during execution [71]. We extend object-level 3D reconstruction into interactive physical proxies and expand structurally feasible states around real-policy failures. This design connects visual reconstruction to policy improvement through a closed loop of object structure, physical predicates, and successful virtual-trajectory feedback.

The most closely related approaches include RoboGSim [72], RoboTwin [73], Real2Render2Real [74], and MimicGen [75]. RoboGSim combines 3DGS with a physics engine for robot-data synthesis and simulation-based evaluation. RoboTwin combines generative digital twins with spatial-relation-aware action-code generation to construct bimanual task data. Real2Render2Real produces rendered training data from object scans and human demonstrations without simulating object dynamics. MimicGen expands datasets by adapting a small set of source demonstrations to new scene contexts.

Our framework uses the current policy’s failure locations to drive local expansion, restricts data feedback through joint task, physics, and safety acceptance, and introduces rejectable low-frequency reference-based recovery during deployment. This comparison concerns feedback sources, candidate acceptance, and deployment interfaces. We do not directly benchmark the four related systems under a unified budget.

2.3. Trajectory extraction from generated video and safe execution

Generative foundation models, video generation models, and 4D world models [7–11] can propose candidate futures conditioned on language and current observations. Generated videos have been used for policy modulation [8], reward construction or data-free imitation [76], and visuomotor learning on real robots [10], reducing the need for additional demonstrations. Visual plausibility, however, does not ensure metric scale, correct contact ordering, dynamic causality, or collision-free execution.

Executable use therefore requires a geometric bridge from video to robot state. Model-based pose estimation [26–32], model-free localization [77–81], semantic spatial localization [82–86], and foundation-model pose estimation [26, 30, 87] progressively reduce dependence on category-specific models. FoundationPose [88] provides a practical route to stable 6D tracking. Nevertheless, pose recovery alone does not determine contact allocation, bimanual end-effector targets, reachability, or dynamically feasible temporal trajectories.

Motion learning, task-parameterized demonstrations, motion priors, and retargeting [89–92] can map recovered motion to a specific robot embodiment. Whole-body control, shared latent spaces, and real-time teleoperation policies [93–95] further reduce embodiment differences. Bimanual deployment faces stronger constraints because both arms share objects and workspace. An apparently valid reference may still violate self-collision, environmental clearance, joint-limit, or contact-consistency constraints.

Risk-aware local motion correction provides an effective way to distinguish nominal control from safety intervention [96]. We further restrict generated video to a rejectable, low-frequency local reference. The system extracts only target image trajectories from generated frames, then connects visual futures to bimanual control through current-state alignment, local action mapping, uncertainty gating, and safety projection. This design addresses the lack of direct action semantics and executability constraints in generated futures while retaining the base VLA’s primary role during normal task steps.

3. Method

The upper, middle, and lower regions correspond to structured policy learning, failure-region data expansion, and local deployment-time recovery, respectively. The upper region restricts inter-arm relations to training-time supervision. The middle region feeds back successful virtual trajectories accepted by the proxies to update the policy. The lower region activates recovery through risk gating, while the base policy remains in control at low-risk steps. Together, these regions form an iterative loop of real demonstrations, proxy-based expansion, and online recovery.

The method is organized around the three interfaces in Fig. 1. Section 3.1 defines the structured bimanual VLA learned from limited demonstrations: the policy receives only synchronized three-view RGB images, language, and robot joint states, and outputs a joint 32-step action chunk for both arms. Object, contact, and stage labels provide training context and do not enter forward inference. Section 3.2 reconstructs objects using calibrated RGB images and object masks, converts the object-level 3DGS representations into Isaac Sim physical proxies for policy rollouts, and expands training data around policy failures. Section 3.3 requests low-frequency generated references under risk conditions and extracts 2D target trajectories from future frames. Current depth and camera calibration map these trajectories

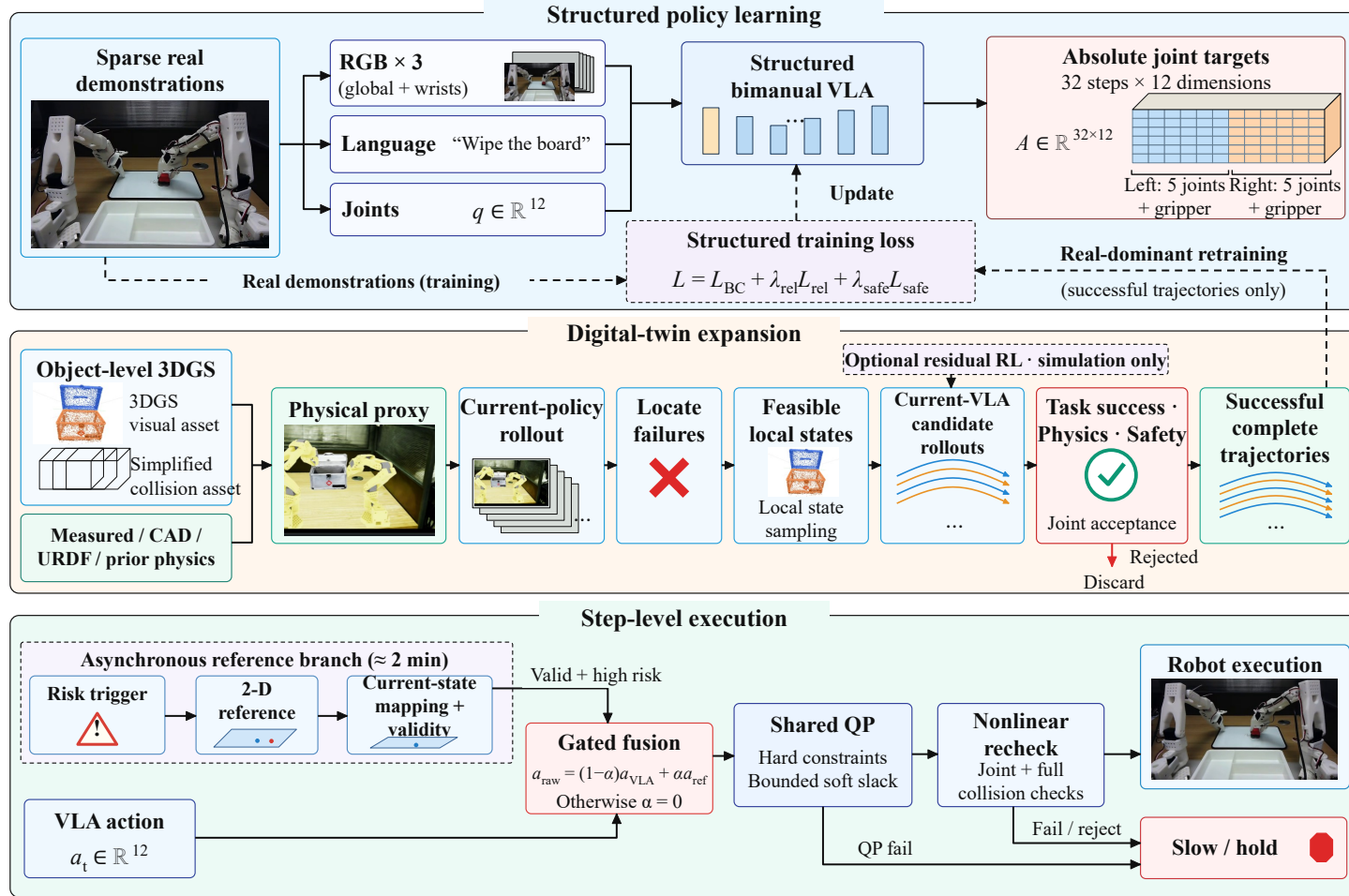


Figure 1: The More from Less closed-loop framework. Structured bimanual VLA learns joint action chunks from sparse real demonstrations. Object-level 3DGS proxies support failure-guided expansion; only successful complete trajectories passing task, physics, and safety acceptance enter real-dominant retraining. At deployment, valid low-frequency references can compensate high-risk steps. Both base and fused actions pass through the shared QP and nonlinear recheck. The schematic layout was initially explored with imagegen and subsequently rebuilt in PowerPoint.

to task-space increments, which task-masked DLS IK converts into bimanual actions. Execution follows gated fusion and the shared QP safety filter.

The three components use visual information differently. The VLA forward inputs in Section 3.1 consist only of a synchronized sample of global, left-wrist, and right-wrist RGB images, language, and bimanual joint states; they exclude depth, object ground truth, and task-stage ground truth. Section 3.2 uses calibrated RGB images and masks for 3DGS reconstruction, with depth used only for metric scale, current-scene point clouds, proxy alignment, and validation. Section 3.3 extracts only 2D image trajectories from generated video; current depth, camera calibration, robot state, and the bimanual roles supplied by the current controller support local mapping. Bimanual self-collision is handled jointly through joint states, URDF collision bodies, forward kinematics, and minimum-distance constraints.

3.1. Structured bimanual VLA

3.1.1. Shared task context and joint bimanual action interface

To unify the input and action interfaces of the three technical modules, let the observation, language task, and joint action of the bimanual robot at discrete time step t be defined as follows.

$$\begin{aligned} o_t &= \{I_t^G, I_t^L, I_t^R, q_t\}, l \in \mathcal{L} \\ A_t &= [A_t^L \ A_t^R] \in \mathbb{R}^{32 \times 12} \end{aligned} \tag{1}$$

In Eq. (1), the three image terms are the global, left-wrist, and right-wrist RGB images acquired at the same control instant, and the joint-state term describes the current bimanual configuration. The task instruction l belongs to the instruction set and conditions the policy separately from the observation. Action chunks are stored along time and joint-channel axes: each arm has a 32×6 chunk, and concatenation along the joint-channel axis yields 32×12 . No concatenation is performed along the time axis. Each single-step action is a 12-dimensional vector of absolute joint position targets, ordered from the left arm to the right arm. Each arm contributes five actuated joints and one gripper component; the field order is given in Table A.4. Depth, object states, and stage labels are excluded from policy forward inputs.

$$\mathcal{D}_{\text{real}} = \{\tau_i \mid i = 1, \dots, N\}, \tau_i = \{(o_t, l, a_t) \mid t = 1, \dots, T_i\} \tag{2}$$

Given the fixed real-demonstration set in Eq. (2), the objective is to extend execution to nearby object poses, contact configurations, and task states.

$$\pi_{\theta}(A_t \mid o_t, l), A_t^r = \{a_{t+h}^r\}_{h=0}^{H_a-1}, H_a = 32 \quad (3)$$

Equation (3) defines the base-policy interface: a synchronized three-view observation produces a 32-step joint action chunk for receding-horizon execution. Geometric relations and safety margins enter training losses, while object, contact, and role annotations remain training context. Object-level 3DGS physical proxies support training-time expansion. Generated-video trajectories provide local deployment references after alignment and validity checks.

3.1.2. Inter-arm relation representation

The difficulty of bimanual manipulation arises not only from high action dimensionality but also from the coupling of the two arms through shared objects, contact states, and relative-pose constraints. Directly concatenating the arms’ actions into a high-dimensional regression vector allows the model to formally output $a_t = [a_t^L, a_t^R]$, but does not explicitly distinguish single-arm action errors from violations of inter-arm relations. We therefore formulate structured bimanual VLA as a policy-interface and training-constraint design, rather than as a new architecture that necessarily modifies the VLA backbone.

A shared context encoder processes the current observations and language instruction, and a joint head predicts both arms’ action chunks. For relation supervision, forward kinematics gives relative end-effector translation and rotation, and collision geometry gives minimum inter-arm distance. Object poses, contact annotations, and step-specific roles describe the fixed training context. These quantities are excluded from policy forward inputs; no additional cross-arm attention layer is introduced.

The experiments use this interface with the Dual-arm VLA backbone. ACT and Diffusion Policy are separate baselines, rather than tested replacements within the same structured configuration.

The relation mapping receives candidate actions and fixed training context separately. Predicted and demonstrated actions are first denormalized into the same absolute joint coordinates, then processed using identical kinematics, geometric queries, and normalization. The resulting relations describe candidate target configurations and are not equivalent to contact ground truth after real-robot execution. Object poses, contact annotations, and role annotations serve as training context and do not enter policy forward inputs.

Given the end-effector transforms obtained through forward kinematics, the relative transform is the inverse left-end-effector transform multiplied

by the right-end-effector transform. Its translation is expressed in the left end-effector frame, and its rotation uses the rotation logarithm on the same local branch. Minimum inter-arm distance is queried using the same collision geometry. The prediction and demonstration branches share coordinate frames, object context, and distance definitions, preventing coordinate transformations or annotation differences from being miscounted as action errors.

3.1.3. Loss-level coordination supervision and safety prior

The relation variables above do not enter the backbone’s forward inference; instead, they constrain predicted actions through the training objective. Let a_t^* denote the demonstrated action and $\hat{a}_t \sim \pi_\theta(\cdot \mid o_t, l)$ the policy prediction. The base imitation loss first constrains the action accuracy of the left and right arms separately:

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_t \left[\|\hat{a}_t^{\text{L}} - a_t^{L,*}\|_1 + \|\hat{a}_t^{\text{R}} - a_t^{R,*}\|_1 \right] \quad (4)$$

Action error alone does not measure relative end-effector geometry. Equation (5) compares the relation descriptor $\tilde{\rho}_{\text{rel}}(q_t, s_t, a_t)$ for predicted and demonstrated targets under the same context s_t . Translation and distance use training-set normalization; orientation uses a local rotation-logarithm mapping. Contact, gripper-stage, and role labels describe the context and cannot be inferred from absolute joint targets alone.

$$\mathcal{L}_{\text{rel}} = \mathbb{E}_t \left[\|W_{\text{rel}} (\tilde{\rho}_{\text{rel}}(q_t, s_t, \hat{a}_t) - \tilde{\rho}_{\text{rel}}(q_t, s_t, a_t^*))\|_2^2 \right] \quad (5)$$

After normalization, Eq. (5) uses the diagonal matrix W_{rel} to adjust component contributions. Because this matrix lies inside the squared norm, the squared diagonal entries are the actual coefficients of the squared residual terms. Their role differs from that of the overall relation weight in Eq. (7). Both branches share the training-set means and positive scales, which remain fixed during validation and testing; a zero scale cannot be used directly as a divisor.

Let ρ denote the unnormalized relation vector, D the fixed diagonal matrix of positive scales, μ the training-set mean, r the relation difference between the branches, and J_ρ the Jacobian of the relation vector with respect to denormalized actions. For a single valid action step, wherever the relation mapping is differentiable, the gradient of Eq. (5) can be written as:

$$\begin{aligned} \tilde{\rho} &= D^{-1} (\rho - \mu), \quad r = \rho(a) - \rho(a^*) \\ \nabla_a \ell_{\text{rel}} &= 2J_\rho^\top D^{-\top} W_{\text{rel}}^\top W_{\text{rel}} D^{-1} r \end{aligned}$$

Gradients pass through action denormalization and the action-dependent relation mapping; demonstrated targets and context labels remain fixed. A contact or role label copied into both branches contributes zero residual and zero action gradient. An independent contact or role penalty would require an action-conditioned mapping, which is not established by the reported implementation details. Geometric distances are generally piecewise differentiable at nearest-point changes. Table A.3 records the remaining loss-configuration limits.

To discourage non-executable or high-risk actions during imitation, we additionally introduce a lightweight safety prior. Let $d_{\text{self}}(q_t, \hat{a}_t)$ denote the minimum bimanual self-collision distance at the predicted target configuration, and $d_{\text{env}}(q_t, s_t, \hat{a}_t)$ the minimum distance from the arms or grippers to environmental obstacles. The safety regularizer is:

$$\begin{aligned} \mathcal{L}_{\text{safe}} = \mathbb{E}_t \bigg[& \text{ReLU}\left(d_{\text{min}} - d_{\text{self}}(q_t, \hat{a}_t)\right)^2 \\ & + \text{ReLU}\left(d_{\text{env,min}} - d_{\text{env}}(q_t, s_t, \hat{a}_t)\right)^2 \bigg]. \end{aligned} \quad (6)$$

Equation (6) imposes a squared hinge penalty on target configurations with clearance below the specified threshold; a term becomes zero once the clearance exceeds that threshold. This training prior evaluates the target configuration and does not cover the complete motion from the current configuration to the target. It therefore cannot replace the shared QP and nonlinear recheck used during deployment.

Combining Eqs. (4)–(6), the structured bimanual policy-learning objective in Section 3.1 is written as follows.

$$\mathcal{L}_{3.1} = \mathcal{L}_{\text{BC}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}} + \lambda_{\text{safe}} \mathcal{L}_{\text{safe}} \quad (7)$$

Equation (7) combines imitation, relation consistency, and the safety prior with nonnegative weights. Each loss is averaged over valid action steps; padding is masked. Figure 2 separates policy inference from the two training branches.

Predicted and demonstrated actions enter the same mapping separately. Training context is shared between the branches and is excluded from the inference path.

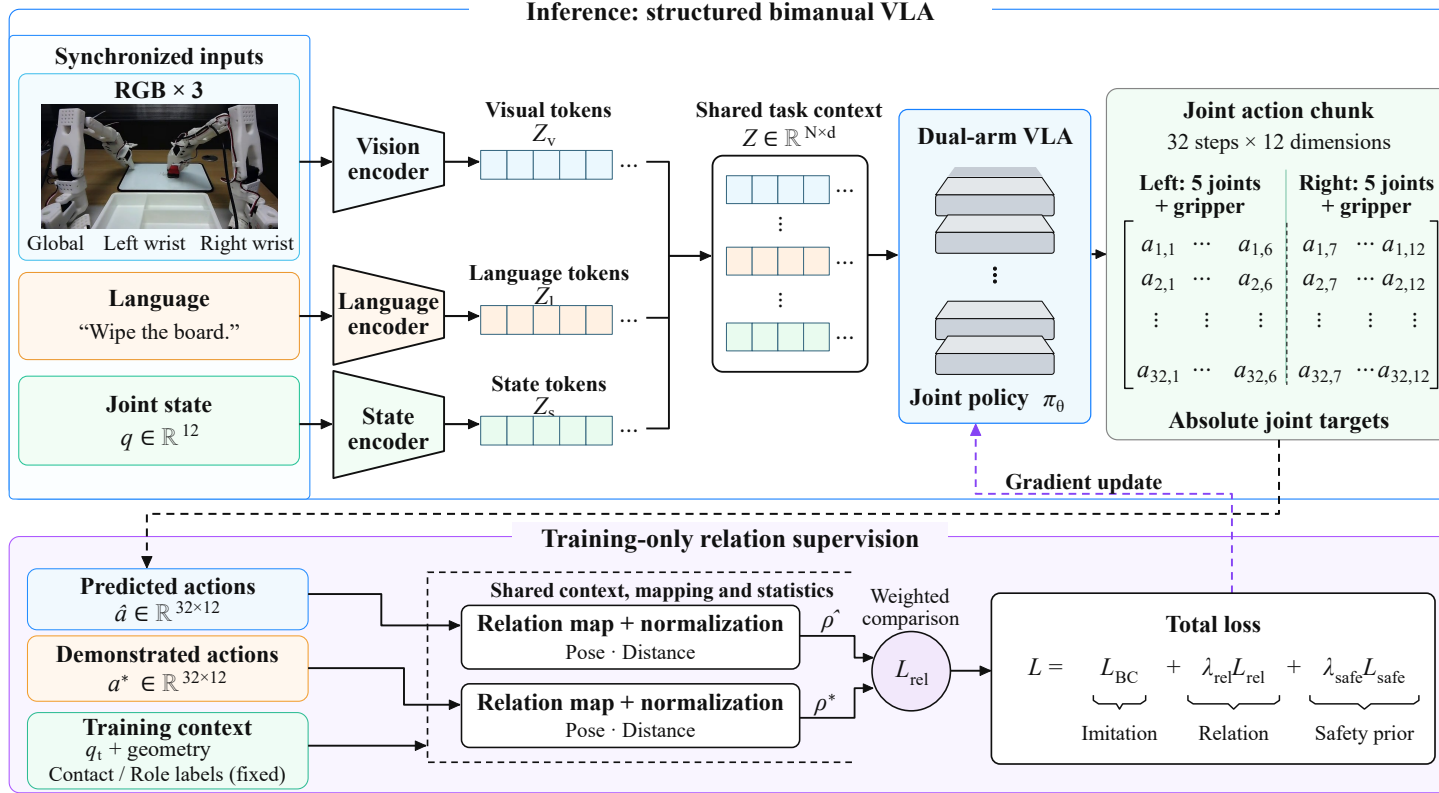


Figure 2: Structured bimanual VLA. Synchronized global and wrist RGB images, language, and joint states produce a joint 32-step chunk of 12-dimensional absolute targets. Predicted and demonstrated actions use the same relation mapping and normalization. The two action-dependent relation maps contain pose and distance. Contact and role labels appear separately in the shared training context; they remain fixed and produce no action gradient. The schematic layout was initially explored with imagegen and subsequently rebuilt in PowerPoint.

3.2. Object-level 3DGS physical proxies and failure-region expansion

The environment in a complex manipulation task is not an arbitrarily varying continuous space, but a constrained system jointly determined by object geometry, contact relations, and articulated structure. If a policy learns action mappings solely from image or semantic features, limited demonstrations must implicitly convey object boundaries, motion directions, joint ranges, and manipulable regions. This can lead to low sample efficiency, unstable action extrapolation, and poor out-of-distribution generalization. We therefore model the scene using a 3D Gaussian representation with object disentanglement and structural-prior constraints, and use it to generate structurally consistent augmented samples to improve learning stability under limited data.

3.2.1. Object-level Gaussian scene representation and structural priors

Given calibrated multiview RGB images $\{I_v\}_{v=1}^V$ and instance masks $\{M_i^v\}$, we decompose the scene into a background model and N manipulable object models. The 3DGS photometric reconstruction objective does not require depth input. The current depth image D_t is used subsequently for metric point-cloud recovery, scale anchoring, physical-proxy alignment, and online obstacle-geometry construction.

$$\mathcal{M} = \{\mathcal{G}_b, (\mathcal{G}_1, \xi_1), \dots, (\mathcal{G}_N, \xi_N)\} \quad (8)$$

In Eq. (8), $\mathcal{G}_b$ is the background Gaussian submodel, $\mathcal{G}_i$ is the Gaussian submodel of object i , and ξ_i denotes its structural degrees of freedom or movable-part parameters. An individual Gaussian primitive is represented as follows.

$$g_j^i = (p_j^i, C_j^i, r_j^i, \sigma_j^i) \quad (9)$$

In Eq. (9), p_j^i is the 3D center, C_j^i the shape covariance, r_j^i the appearance attributes, and σ_j^i the opacity. Object-level decomposition makes foreground objects, the background, and distinct manipulable objects independent in parameter space.

Instance segmentation restricts the supervision region of each submodel. Let Λ_i^v be the pixel support of object i in view v , and Ω the full image domain. The background support is then:

$$\Lambda_b^v = \Omega \setminus \bigcup_{i=1}^N \Lambda_i^v \quad (10)$$

Using the support domains defined in Eq. (10), object and background models receive rendering supervision within their respective supports. The object-level reconstruction loss is written as follows.

The valid index set in Eq. (11) contains the background and objects with nonempty support in the current view: k belongs to $\{b, 1, \dots, N\}$, and the corresponding support size is greater than zero. This set provides a unified notation for the background and object terms below.

$$L_{\text{geo}} = \sum_{v=1}^V \sum_{k \in \mathcal{K}_v} \frac{1}{|\Lambda_k^v|} \sum_{u \in \Lambda_k^v} d_{\text{RGB}}(R_v(\mathcal{G}_k)(u), I_v(u)) \quad (11)$$

Equation (11) sums over nonempty support domains; an empty mask does not produce a division-by-zero term. Rendering is evaluated from view v , u denotes an image pixel, and $d_{\text{RGB}}(\cdot, \cdot)$ is an RGB-domain photometric or perceptual reconstruction error, not a depth loss. A fully occluded object contributes no photometric term in that view. If every support domain is empty, the view contributes nothing to this loss. Articulated-structure parameters are specified separately using hinge axes, sliding directions, and motion ranges.

3.2.2. From 3DGS to Isaac Sim physical proxies

Visual and collision assets are built separately at the object level. Instance masks, opacity, and clustering distance remove background and outlier Gaussians. The retained means and scales are converted to a triangular mesh, simplified, and written to USDZ for collision queries. Gaussian or original appearance assets are used for rendering. Thin-walled, transparent, or heavily occluded objects can instead use conservative collision geometry from CAD, category priors, or measurements; these sources are distinct from the visual reconstruction.

Mass, inertia, friction, restitution, and joint parameters come from measurements, CAD/URDF, or material priors. Short push-and-slide tests, grasp-slip tests, joint responses, and simulation-to-real pose comparisons are used for diagnosis. Training may randomize parameters within calibration intervals, while evaluation uses fixed settings. Table A.6 lists the available construction settings.

The physical proxy for object k is denoted by:

$$\mathcal{P}_k = \{M_k^{\text{vis}}, C_k^{\text{col}}, m_k, I_k, \mu_{\text{fric}}, e_{\text{rest}}, \xi_k\} \quad (12)$$

In Eq. (12), the visual asset and collision geometry support rendering and contact computation, respectively. Mass, inertia, friction coefficient, coefficient of restitution, and structural degrees of freedom are obtained from recorded measurements or priors and configured in Isaac Sim. This representation connects object-level visual reconstruction to the physical proxies required for executable policy rollouts, without interpreting rendering quality itself as dynamic accuracy.

An articulated object is first divided into independent rigid parts. A record file then specifies the splitting plane, hinge point, hinge axis, fixed part, and joint limits. The existing hinge pipeline uses a default example range of 0° – 90° with part A fixed. When CAD, URDF, or manual annotations are available, their joint axes, limits, damping, and stiffness are used directly. Collision assets are generated separately for each part, and the parts are connected by revolute or prismatic joints. These physical parameters come from external records and are not inferred automatically by 3DGS.

The physical proxy does not aim to reproduce real dynamics completely. Instead, it satisfies three requirements directly relevant to policy rollouts: rendered appearance maintains visual-distribution consistency, collision geometry prevents penetration and incorrect contact, and articulated joints constrain the task-feasible domain. Only virtual trajectories meeting these requirements are fed back for training.

For objects with movable parts, we introduce low-dimensional structural parameters. Hinged objects are described by a rotation axis u_k , a reference point on that axis p_k , and an angle θ_k . Sliding objects are described by a sliding direction v_k and displacement d_k . These parameters can be obtained from manual annotations, CAD/URDF priors, video-trajectory fitting, or estimates of multiview pose changes.

These structural degrees of freedom are encoded as joint constraints in Isaac Sim, restricting object-state changes to physically feasible space. The digital-twin environment thus retains the visual detail of 3DGS while acquiring the collision bodies, joints, mass, and contact properties required by the physics engine.

3.2.3. Failure-region expansion and residual reinforcement-learning refinement

Failure regions are defined in task-stage and object coordinates, rather than by image similarity alone. The state of an individual policy rollout is written as follows.

$$x = (s, T_{\text{obj}}, \xi, T_{\text{ee}}^{\text{L}}, T_{\text{ee}}^{\text{R}}, c, u, e) \quad (13)$$

In Eq. (13), s denotes the task stage, T_{obj} the target object’s 6D pose, ξ the articulated or sliding state, T_{ee}^{L} and T_{ee}^{R} the two end-effector poses, c the contact mode, u policy uncertainty, and e deviation from the target or reference trajectory. The failure set is defined accordingly below.

$$\mathcal{F} = \{x \mid F_{\text{stage}}(x) = 1 \vee C_{\text{safe}}(x) = 0 \vee u > \tau_u \vee e > \tau_e\} \quad (14)$$

In Eq. (14), the stage-failure predicate is 1 when the current stage terminates without satisfying its success condition, times out, or incurs an unrecoverable deviation. The safety predicate is 1 when there is no environment collision, self-collision, joint-limit violation, or object penetration. Uncertainty and deviation thresholds serve only to locate neighborhoods for expansion and are frozen on the validation set before use in testing.

When generating candidate tasks around a failed state, the system locally perturbs T_{obj} , ξ , T_{ee}^{L} , T_{ee}^{R} , and the target region in object coordinates while preserving constraints on grasp contact, hinge limits, sliding directions, tabletop support, and bimanual reachability. Candidate states are local boundary samples within the object’s structurally feasible domain, rather than unconstrained random noise. Successful policy rollouts can therefore supplement states most poorly covered by the current policy without substantially increasing physically inconsistent training samples.

Rollout acceptance follows the task stage: grasping, transport, placement, and structural interaction check contact, lift, target region, pose, or joint progress. Environment collision, self-collision, joint-limit violation, or object penetration rejects a candidate. Threshold selection uses validation data or physical tolerances before evaluation; Table A.6 describes the checks and available settings.

The lower half of Fig. 3 summarizes task-boundary expansion guided by failure regions. Rather than uniformly expanding the full state space, the system generates structurally feasible candidate tasks near the current policy’s failed states. Task-success predicates, safety predicates, and structural-consistency filtering determine whether trajectories are fed back for training.

Original demonstrations typically cover state–action pairs close to only a few expert trajectories, leaving limited coverage of different opening angles, sliding positions, and contact states. Direct random perturbation of states or actions can increase dataset size, but the resulting samples need not satisfy geometric relations or motion constraints and may even contain penetration, collision, or actions in incorrect directions. We therefore use the structured Gaussian scene to generate augmented data within feasible structural space.

To prevent virtual-data expansion from degenerating into undirected random sampling, the current policy is first deployed in the structured digital-twin environment. Key states in failed trajectories are recorded, including object poses, movable-part states, bimanual end-effector poses, contact relations, and target-region deviations. Candidate task states are then generated around these failures within the object’s structurally feasible domain, rather than uniformly throughout the full state space. New samples thus concentrate on physically feasible boundary regions that the current policy has not yet mastered.

The structured 3D Gaussian environment therefore serves two roles: it provides interactive digital-twin assets at the object and articulated-structure levels, and it defines the constrained space for locating failure regions and generating candidate states. A virtual rollout is fed back into the training set only if it simultaneously satisfies structural ranges, collision checks, contact relations, and task-completion conditions.

Let $x_f \in \mathcal{F}$ be a failed state produced by the current policy, and i an object whose structural coordinate at that state is $q_{i,f}$. Equation (15) locally samples candidates within the object’s valid range.

$$q'_i \sim \nu_i(\cdot \mid q_{i,f}, \varepsilon_i), q'_i \in [q_i^-, q_i^+] \quad (15)$$

The structured scene model maps the sampled structural coordinates and the source failed state to the candidate initial state in Eq. (16).

$$x'_0 = \Phi_{\text{scene}}(x_f, q'_1, \dots, q'_N; \mathcal{M}) \quad (16)$$

The system then rolls out the current policy from the candidate state without introducing a separate action generator.

$$\begin{aligned} z'_t &= E_\theta(o'_t, l), a'_t \sim \pi_\theta(\cdot \mid z'_t) \\ \tau' &= \text{Rollout}(\pi_\theta, x'_0, \mathcal{M}) \end{aligned} \quad (17)$$

Equation (17) uses the current VLA as the base rollout policy. Object and initial states are perturbed within failure neighborhoods, while actions are sampled from the policy. Optional residual RL acts only as a local explorer in the digital twin to increase candidate diversity. Regardless of the explorer used, only complete trajectories that pass task, physics, and safety acceptance enter the virtual-success set.

$$\begin{aligned}\Gamma(\tau') &= S_{\text{task}}(\tau')C_{\text{safe}}(\tau')P_{\text{phys}}(\tau') = 1 \\ \mathcal{D}_{\text{virt}}^+ &= \{\tau' : \Gamma(\tau') = 1\}\end{aligned}\tag{18}$$

The accepted virtual set in Eq. (18) is mixed with real demonstrations through the two explicit expectation terms in Eq. (19), rather than by taking an expectation over an unweighted union:

$$\begin{aligned}\theta^* &= \arg \min_{\theta} \left[\omega \mathbb{E}_{\tau \sim \mathcal{D}_{\text{real}}} \mathcal{J}(\tau; \theta) \right. \\ &\quad \left. + (1 - \omega) \mathbb{E}_{\tau' \sim \mathcal{D}_{\text{virt}}^+} \bar{w}(\tau') \mathcal{J}(\tau'; \theta) \right], \\ \omega &\in \left(\frac{1}{2}, 1 \right].\end{aligned}\tag{19}$$

The per-trajectory objective in Eq. (19) is Eq. (7) averaged over valid action steps. Real-data dominance is a condition on objective coefficients: the real-data weight must exceed one-half, virtual quality weights must be nonnegative, and the virtual mixture weight multiplied by the mean virtual quality weight must be smaller than the real mixture weight. Neither increasing the number of real trajectories nor constraining the mixture coefficient alone replaces this condition, which also does not ensure that real data dominate the gradient at every step. If no virtual trajectories have been accepted, the virtual expectation is undefined and training uses only the real-data term.

Visual assets provide appearance rendering, while simplified collision bodies and measured or prior physical parameters support interaction. In the lower part, failed states serve as centers for local sampling. Candidates form the next round of training data only after completing the task and passing physics and safety acceptance.

The initial failure set in Algorithm 1 comes from the initial policy evaluated on training/adaptation initial states. Each round repeats failure discovery with the updated policy. Only accepted complete trajectories enter the accumulated virtual set; failed trajectories locate sampling regions. M is the

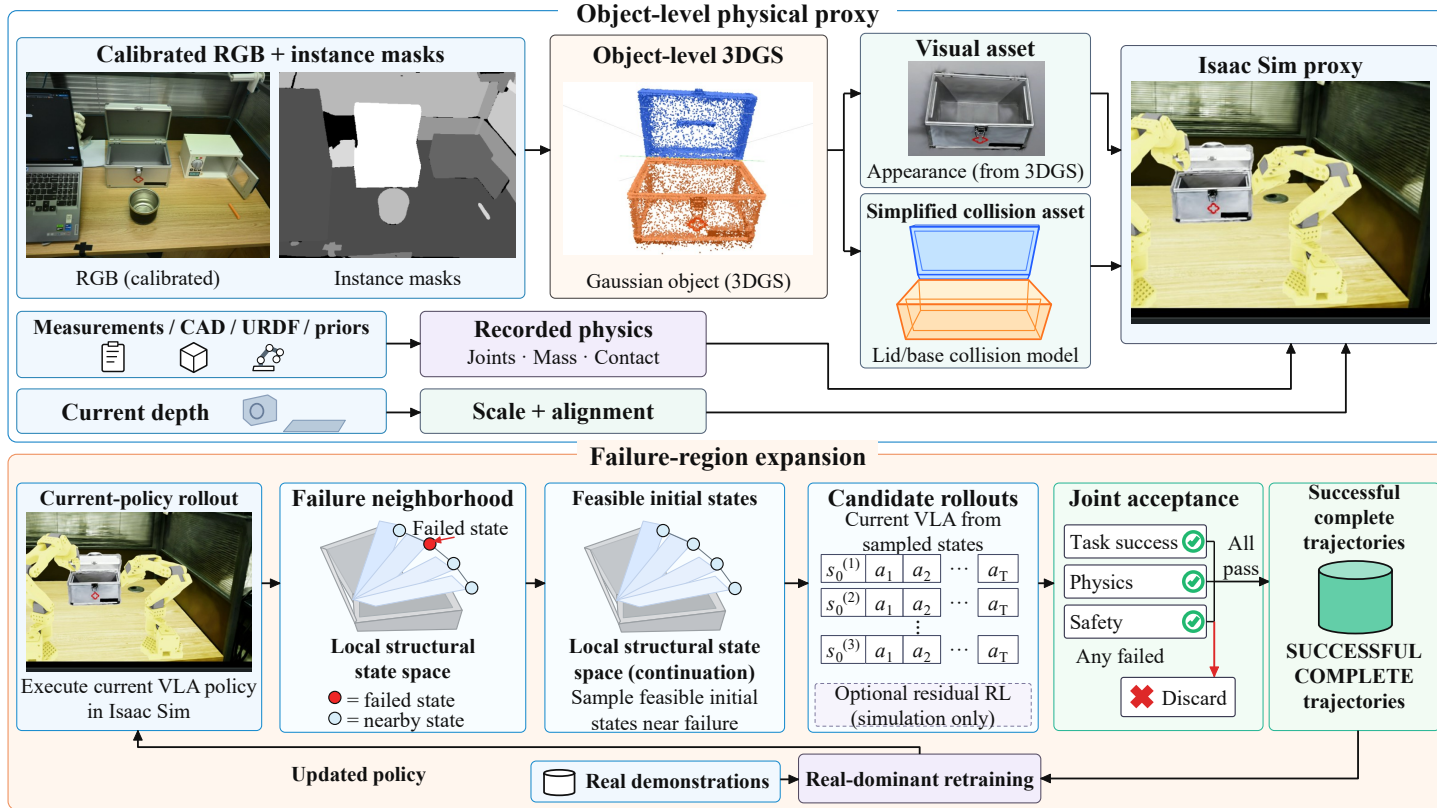


Figure 3: Object-level proxies and failure-guided expansion. Top: calibrated RGB and masks yield 3DGS visual assets, combined with simplified collision assets and recorded physics in Isaac Sim. Bottom: failures locate feasible sampling neighborhoods. Only successful complete trajectories passing joint acceptance enter real-dominant retraining. Optional residual RL proposes simulation candidates subject to the same acceptance; it is not deployed on the robot. The schematic layout was initially explored with imagegen and subsequently rebuilt in PowerPoint.

prescribed iteration cap. Tables 2 and 3 compare expansion settings and their endpoint results.

Algorithm 1. Failure-region-guided digital-twin expansion

Input: D_{real} , initial policy π_θ , object-level digital twin E_G , iteration count M

- 1: Train π_θ on D_{real} using structured bimanual VLA loss.
- 2: for $m = 1 \dots M$ do
- 3: Roll out the current VLA from the training/adaptation initial-state set to discover failures; keep held-out test episodes excluded.
- 4: Collect failure states F_m with stage, pose, contact, uncertainty and task predicate logs.
- 5: Sample candidates x_c around each failure state under structural, collision, reachability and semantic constraints.
- 6: Roll out candidates with the current VLA or optional simulation-only residual explorer; add only complete trajectories accepted by Eq. (18) to Dvirt+.
- 7: Retrain the VLA on D_{real} and Dvirt+ using Eq. (19); do not deploy the residual policy on the robot.
- 8: end for; return enhanced policy π_θ^+ .

Following failure-region expansion, residual reinforcement learning (residual RL) serves only as an optional local explorer in Isaac Sim, generating candidate trajectories for contact, alignment, and transport stages. It does not directly control the real robot.

The structured VLA supplies base simulation actions, while the residual policy learns bounded local corrections. Every residual rollout undergoes the same task, physics, and safety acceptance as in Eq. (18). Rejected trajectories are discarded; accepted trajectories form the accepted set and are fed back to retrain the VLA.

Residual control combines the base VLA action with a magnitude-bounded residual action:

$$\begin{aligned} a_t &= a_t^{\text{vla}} + \text{clip}(\delta a_t, -\epsilon_a, \epsilon_a), \\ \delta a_t &\sim \pi_{\text{RL}}(\cdot \mid x_t, z_t). \end{aligned} \tag{20}$$

Following the residual-control definition in Eq. (20), the reward combines task success, task progress, stable contact, action smoothness, and safety penalties:

$$\begin{aligned} r_t &= w_s R_{\text{success}} + w_p R_{\text{progress}} + w_c R_{\text{contact}} \\ &\quad - w_{\text{col}} I_{\text{collision}} - w_j \|a_t - a_{t-1}\|_2^2 - w_o R_{\text{out}}. \end{aligned} \tag{21}$$

The reward in Eq. (21) is used exclusively for residual exploration in the digital twin. Residual trajectories accepted through Eq. (18) form the accepted RL-trajectory set. This set is merged into the accepted virtual set of Eq. (19), after which the VLA is retrained. Real-robot deployment runs only the updated VLA, compensation gate, and safety filter, without the residual policy. Thus, real-robot differences in the experimental row labeled “RL trajectory feedback” arise from the VLA trained with the added data, not from real-robot residual control.

3.3. Uncertainty-aware step-level trajectory compensation

3.3.1. Trajectory extraction from generated video and local action mapping

The reference trajectories in this section are neither high-frequency online planning outputs nor final robot commands. Future-video generation and image-trajectory extraction run asynchronously at low frequency after task or step transitions and failure detection. While waiting, the control loop continues to execute base VLA commands processed by the shared safety filter. A timestamped reference enters local fusion only after realignment to the current state, validity checks, and task-masked damped least-squares (DLS) inverse kinematics.

The structured bimanual policy in Section 3.1 remains a conditional policy model, while the 3DGS physical proxies and failure-region expansion in Section 3.2 primarily operate during training. Both improve the coverage of the base policy, yet deployment can still encounter step-level deviations caused by unseen contact states, local occlusion, or object slip. We therefore introduce generative reference trajectories during deployment as compensation priors for local steps that the VLA cannot complete reliably.

From a modeling perspective, step-level compensation does not require the VLA to predict long horizons independently. Instead, it introduces a low-frequency video trajectory $\tilde{\tau}_{t+1:t+H}^{\text{vid}}$ induced by a distribution over visual futures. Future frames provide visual evidence only for the temporal motion of a target or task-relevant region, with the following generative distribution.

$$p(v_{t+1:t+H} | o_t, l) = \prod_{\tau=1}^H p(v_{t+\tau} | v_{t+1:t+\tau-1}, o_t, l) \quad (22)$$

In Eq. (22), H is the reference horizon, and $v_{t+\tau}$ is the generated future frame at offset τ . This sequence is used to extract a target image trajectory. It is neither executed directly nor interpreted as an object 6D pose, contact point, or bimanual end-effector target.

The system then applies the trajectory-extraction operator Φ_{traj} to convert the generated frame sequence into a target trajectory in the image plane.

$$\tilde{\tau}_{t+1:t+H}^{\text{vid}} = \Phi_{\text{traj}}(v_{t+1:t+H}) = \{\tilde{p}_{t+\tau}^{\text{img}}\}_{\tau=1}^H \quad (23)$$

In Eq. (23), $\tilde{p}_{t+\tau}^{\text{img}}$ denotes the image coordinates of the target or task-relevant region in a generated frame, and $\tilde{\tau}_{t+1:t+H}^{\text{vid}}$ the low-frequency video trajectory formed by those coordinates. The generated video supplies only this trajectory; it does not directly recover object 6D poses, contact points, or bimanual end-effector references.

Image-trajectory extraction and control mapping use separate interfaces. Framewise masks of the target object, tool, or task region yield image coordinates and tracking confidence. Continuity checks, outlier removal, and temporal smoothing then produce a local reference. Low confidence, prolonged occlusion, identity switches, or trajectory discontinuities cause the reference to be rejected.

Generated frames provide only 2D pixel increments. Under a locally constant current-target-depth approximation, Eq. (24) maps pixel increments to translations within the camera imaging plane, which camera extrinsics then transform into the robot base frame. This mapping does not recover unobserved motion along the depth direction or end-effector rotation. Both gripper targets are taken from the current VLA action.

Current-state alignment and local task mapping produce end-effector displacement references for both arms in the base frame. Given the current joint state, Eq. (24) uses task-selection matrices to mask components that are unobserved in the task or cannot be tracked unconditionally by a 5-DoF arm. Damped least squares then solves for the joint increments of both arms jointly.

$$\begin{aligned} \Delta p_{t+\tau} &= [\Delta u_{t+\tau}, \Delta v_{t+\tau}]^\top, \\ \Delta x_{t+\tau}^{\text{B}} &= R_{\text{BC}}[Z_t \Delta u_{t+\tau}/f_x, Z_t \Delta v_{t+\tau}/f_y, 0]^\top, \\ \Delta q_t^* &= \arg \min_{\Delta q} \left\{ \sum_{r \in \{\text{L}, \text{R}\}} \|S_r(J_r(q_t) \Delta q - \Delta x_{\text{ref}}^r)\|_2^2 \right. \\ &\quad \left. + \lambda_{\text{DLS}}^2 \|\Delta q\|_2^2 \right\}. \end{aligned} \quad (24)$$

The first line of Eq. (24) gives the pixel increment of the generated trajectory. The second computes a translation increment from current target depth,

camera intrinsics, and camera-to-base extrinsics. The third obtains a joint correction using task-selection matrices and the left- and right-arm Jacobians. The retained Isaac Lab source code specifies a default damping parameter of 0.01 and uses its square in the solver; the regularization coefficient is therefore written as the squared damping parameter. This default is not a substitute for the original experimental run configuration. The independent task components selected for each arm must be compatible with its five actuated degrees of freedom. The resulting joint increments are added to the current state and assembled into a 12-dimensional absolute position reference according to Table A.4, which must still pass action-boundary and shared-QP checks.

The joint increment contains ten actuated joints across both arms and excludes the grippers. Each end-effector Jacobian is 6×10 , with zeros in the columns corresponding to the other arm. Each task-selection matrix is $k \times 6$, where k is the number of task components selected for that arm. The active arm uses the aligned image-translation target, while the supporting arm uses the current contact-maintenance conditions. Placeholder zeros for unobserved rotation are masked out and must not be interpreted as orientation-preservation constraints. The number of independent selected components for each arm does not exceed its five actuated degrees of freedom; the effective rank of the current Jacobian and the task residual must still be checked. The reference is invalid if masks or support conditions are unspecified, depth is invalid, the target is unreachable, or the residual exceeds its limit.

The masked Jacobians and targets of the left and right arms are stacked row-wise and denoted by $\bar{J}$ and $\bar{x}$, respectively. The local solution of Eq. (24) and the construction of the reference action are given below. Here, q is the column vector of current arm-joint positions; the left- and right-arm increments are the corresponding subvectors of the optimum; and g denotes the current VLA gripper targets. The ordering follows Table A.4:

$$\Delta q^* = \left(\bar{J}^\top \bar{J} + \lambda_{\text{DLS}}^2 I_{10} \right)^{-1} \bar{J}^\top \bar{x}$$

$$a_{\text{ref}} = \left[(q_L + \Delta q_L)^\top, g_L, (q_R + \Delta q_R)^\top, g_R \right]^\top$$

The online path in the figure has a single robot-execution outlet. Once current-state mapping and validity checks are complete, current risk determines the reference branch’s fusion weight. Both base commands and fused commands pass through the shared QP and nonlinear recheck.

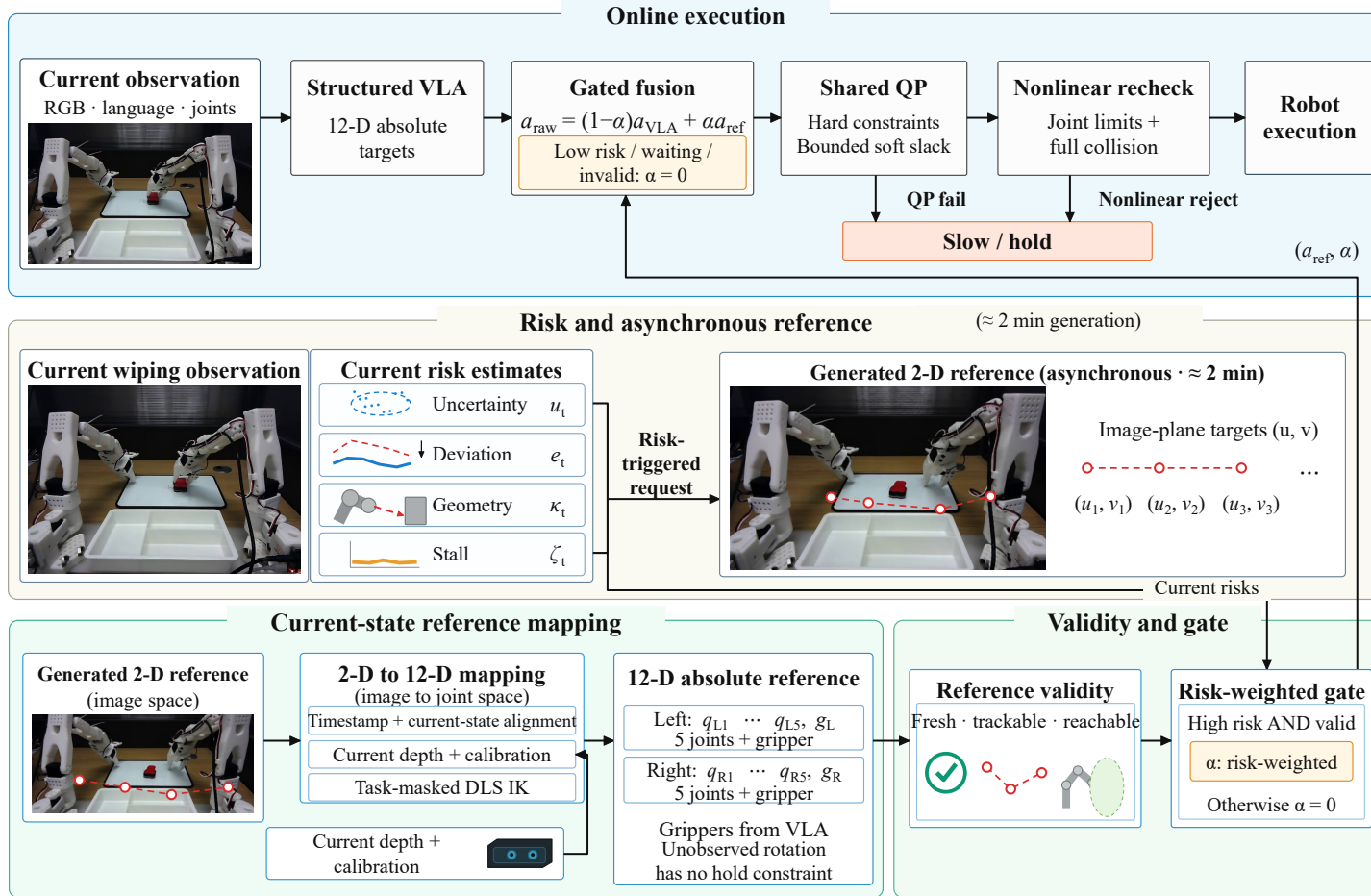


Figure 4: Step-level compensation and shared safety filtering. High risk requests an asynchronous 2D reference. Current-state alignment, depth and calibration, task-masked DLS IK, and validity checks precede gated fusion. Low risk, waiting, or invalidity sets the fusion weight to zero. All actions pass through the shared QP and nonlinear recheck; failure causes slowdown or hold. Grippers inherit VLA targets; unobserved orientation has no exact-preservation guarantee. The schematic layout was initially explored with imagedgen and subsequently rebuilt in PowerPoint.

Future video therefore provides only a rejectable, low-frequency 2D image-trajectory prior. A single generation takes approximately 2 minutes, making this an asynchronous local-recovery process that allows waiting, rather than real-time video servoing. While waiting, the base VLA continues to supply commands processed by the safety filter. Fusion occurs only when risk exceeds its threshold, the returned reference remains fresh, and trajectory, depth, and IK checks all pass. Invalid or expired references are rejected. QP infeasibility, excessive soft-constraint slack, or failure of the nonlinear recheck causes slowdown or hold, as shown in Fig. 4.

The asynchronous sequence is request, return, current-state alignment, use, and invalidation. Reference age starts at the observation supplied to the generator and is not reset on return. Every returned reference is aligned and checked before use. The approximately two-minute generation latency is separate from the control period; the base VLA continues through the shared safety filter while waiting.

Kling AI 3.0 generates candidate future frames from the current RGB image and language instruction. Equation (25) extracts image-plane targets and validity information. Table A.9 distinguishes the saved-video and offline-extraction settings from the online configuration.

$$\left(\tilde{p}_{t+\tau}^{\text{img}}, c_{t+\tau}^{\text{trk}}\right) = \Phi_{\text{trk}}\left(I_{t+\tau}^{\text{gen}}, M_{t+\tau}\right) \quad (25)$$

Equation (25) gives the framewise implementation of Eq. (23): the centroid of the target or task-region mask is computed in each generated frame and accompanied by tracking confidence. Valid coordinates undergo normalization, outlier rejection, and temporal smoothing. Missing masks, identity switches, persistently low confidence, or trajectory jumps cause the entire reference window to be rejected.

The smoothed image trajectory is mapped using Eq. (24). Current depth supplies scale only for target pixels in the current real observation; object 6D poses, contact points, and end-effector orientations are not recovered from generated frames. Camera extrinsics transform translation increments into the robot base frame, and task-selection matrices determine which components each arm can track. References that cannot be aligned, are unreachable, or violate current grasp relations are rejected.

3.3.2. Uncertainty triggering and reference-trajectory fusion

A risk-threshold crossing, step transition, or detected failure first triggers an asynchronous candidate request carrying a state timestamp. The returned result must be realigned to the current observation and pass trajectory, depth, IK, action-boundary, and collision checks in sequence. The binary gate enables fusion only when both the risk condition and reference validity hold.

Let the VLA output the base action. Action uncertainty is defined as the normalized dispersion of repeated stochastic action-chunk predictions for the same observation. Step deviation is the normalized error of the current task quantity relative to the stage tolerance. Geometric risk is the largest normalized shortfall among collision-clearance and joint-margin measures, and stage stagnation denotes insufficient task progress within a fixed window. The reference-validity indicator is 1 only when trajectory, depth, timestamp, and IK checks all pass. Equation (26) uses these quantities to define fusion activation and its continuous weight.

$$\begin{aligned} b_t &= \mathbf{1}[(u_t > \tau_u) \vee (e_t > \tau_e) \vee (\kappa_t > \tau_\kappa) \vee (\zeta_t > \tau_\zeta)]g_t^{\text{ref}} \\ \alpha_t &= b_t\sigma(w_u\tilde{u}_t + w_e\tilde{e}_t + w_\kappa\tilde{\kappa}_t + w_\zeta\tilde{\zeta}_t - \tau_\alpha) \end{aligned} \quad (26)$$

Equation (26) enables fusion only when a risk threshold is crossed and the reference is valid. Within this window, normalized risks set the continuous weight. Otherwise, the hard gate sets the reference weight to zero.

Action fusion uses a convex combination in the same command space:

$$a_t^{\text{raw}} = (1 - \alpha_t)a_t^{\text{vla}} + \alpha_t a_t^{\text{ref}} \quad (27)$$

In Eq. (27), the reference action is obtained through the task-masked DLS IK of Eq. (24), with joint increments converted to 12-dimensional absolute joint position targets relative to the current state. Base and reference actions are first denormalized into the same robot command space and then combined using the gated weight. Because the reference and base targets for both grippers are identical, they remain unchanged through fusion, although subsequent safety filtering may still modify the commands. Thus, only the two gripper targets are explicitly inherited; end-effector orientations not constrained by the task masks may still change as arm joints move. Exact orientation preservation would require additional feasible orientation constraints and is not claimed as an existing property of Eq. (24).

3.3.3. Safety-constrained execution

Every candidate 12-dimensional joint target passes through the shared safety filter. Environmental distances use current-depth geometry; self-collision distances use joint states, forward kinematics, and link collision bodies. The QP minimizes command changes subject to hard constraints and bounded soft slack. A nonlinear recheck precedes command release.

$$a_t^{\text{safe}} = \arg \min_{a \in \mathcal{C}_t} \|a - a_t^{\text{raw}}\|_2^2 \quad (28)$$

Equation (28) seeks the absolute joint position target closest to the original fused target within the nominal feasible set. Equations (29)–(31) provide a local convex implementation, with Eq. (30) defining command boundaries. Joint ranges, single-step changes, and minimum collision distances are hard constraints, while recoverable contact or task deviations may be treated as soft constraints with bounded slack. The solution is rejected if hard constraints are infeasible, slack exceeds its bounds, or the nonlinear recheck fails.

Equations (29)–(31) define the minimum-correction objective, absolute joint position boundaries, and hard/soft locally linearized constraints, respectively. Hard constraints use no slack. Each soft-constraint slack variable is nonnegative and has a fixed upper bound. All thresholds and upper bounds are frozen before testing; fallback and reference-rejection details are given in Tables A.5 and A.8.

$$(\Delta a_t^*, s_t^{\text{qp},*}) = \arg \min_{\Delta a, s^{\text{qp}} \geq 0} \frac{1}{2} \|\Delta a\|_W^2 + \frac{\lambda_{\text{qp}}}{2} \|s^{\text{qp}}\|_2^2 \quad (29)$$

$$a_t^{\text{safe}} = a_t^{\text{raw}} + \Delta a_t^*$$

$$\begin{aligned} q_{\min} &\leq a_t^{\text{raw}} + \Delta a \leq q_{\max} \\ -\delta_q &\leq a_t^{\text{raw}} + \Delta a - q_t \leq \delta_q \\ -\delta_a &\leq a_t^{\text{raw}} + \Delta a - a_{t-1}^{\text{safe}} \leq \delta_a \end{aligned} \quad (30)$$

$$\begin{aligned} G_t^{\text{hard}} \Delta a &\geq h_t^{\text{hard}} \\ G_t^{\text{soft}} \Delta a + s^{\text{qp}} &\geq h_t^{\text{soft}}, 0 \leq s^{\text{qp}} \leq s_{\max}^{\text{qp}} \end{aligned} \quad (31)$$

In Eqs. (29)–(31), action correction is performed in 12-dimensional absolute joint-target coordinates. Positive-definite weights penalize command modifications, and soft-constraint slack is restricted to recoverable task/contact deviations. Equation (30) constrains absolute targets and single-step changes.

The hard-constraint matrix covers joint ranges and minimum environmental and inter-arm distances; the soft-constraint matrix covers local conditions that may be relaxed briefly within explicit upper bounds. The safety sequence is gate-weight smoothing, hard gating, candidate-command smoothing, QP solution, nonlinear rechecking, and command release. Waiting or invalid references are still forced to zero by the hard gate after weight smoothing. When command smoothing is enabled, the smoothed candidate becomes the QP’s raw target, and constraints are relinearized around that candidate. The command to be sent is not modified after the recheck.

If the QP is infeasible, a soft-constraint slack exceeds its bound, or the nonlinear recheck fails, the candidate is rejected and execution falls back to slowing or holding. The most recently accepted command is only a fallback candidate and must be rechecked after changes in the environment or current state. Waiting for a reference or reference invalidity only sets the reference weight to zero and does not itself constitute safety-filter failure.

The execution path therefore has one command filter: reference mapping and gated fusion feed the same QP and nonlinear recheck as the base VLA. Reference rejection changes the fusion weight; command rejection triggers slow or hold behavior.

4. Experiments and analysis

Fig. 5 shows the six bimanual tasks and representative RGB observations. Each three-view group comprises global, left-wrist, and right-wrist images; the wipe-and-pour composite task is illustrated by its constituent stages. The experimental data also include sparse real demonstrations, auxiliary depth records, multiview videos, object masks, and object-level 3DGS scenes. RGB images, language, and robot states are used to train the base policy; depth is used only for the physical proxy, current-scene scale alignment, and safety geometry. Tables A.1 and A.2 describe the sources, uses, and episode-separation principles for these data.

4.1. Experimental setup

4.1.1. Tasks and experimental setup

The experimental tasks match Fig. 5 and comprise bimanual water pouring, microwave opening, medicine-box opening, board wiping, the wipe-and-pour composite task, and the hold-and-wipe-board cooperative task. These six tasks cover grasping, transport, pouring, opening and closing, wiping, and

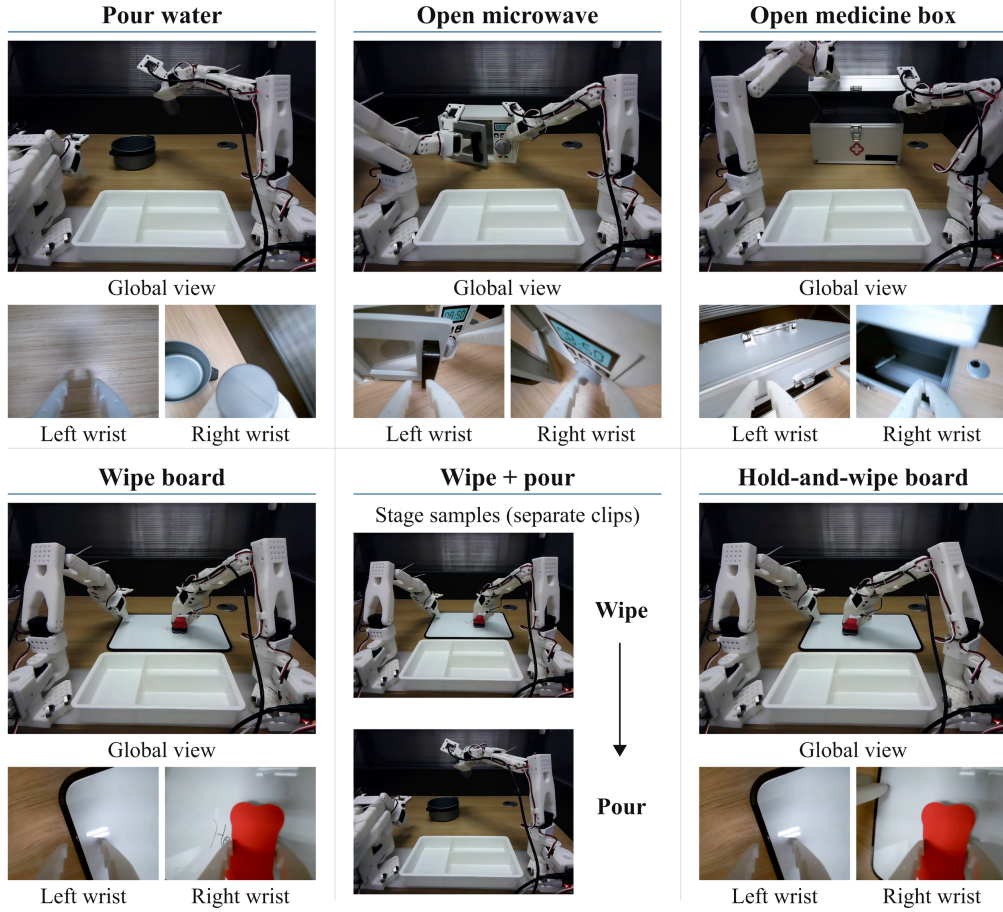


Figure 5: Examples of six bimanual tasks and multiview observations. Frames in each three-view group are sampled at the same relative time in the source videos. Separate clips illustrate the constituent stages of the wipe-and-pour composite task. The wipe-board and hold-and-wipe-board tasks use different times from the same recording to show wiping and cooperative board stabilization, respectively. These examples do not represent six independent complete evaluation episodes.

cross-stage coordination, and are used to evaluate base execution, local recovery, and robustness. Tables A.1 and A.2 describe the data composition and separation principles.

“Sparse demonstrations” refers here to a fixed six-task collection concentrated around limited initial states and contact trajectories. Methods share the task-level split, with all modalities from an episode kept together. This ex-

periment examines coverage around that collection, rather than performance as a function of demonstration count.

Virtual-environment and real-robot evaluations provide different forms of evidence. The virtual environment controls initial states, friction, object joints, and perturbation ranges to test whether failure-region expansion outperforms uniform random augmentation. Real-robot evaluation tests whether these expansions translate into practical success and recovery rates. For methods using reference trajectories, the video generator produces future visual sequences from the same current RGB observations and instructions. The same trajectory extraction, current-state alignment, and local action-mapping procedure then yields reference actions. Thus, trajectory compensation, the video-trajectory baseline, and the hand-crafted-waypoint control share consistent task inputs.

Five independently trained policies are each evaluated with three environment seeds and the same initial-state set, yielding 15 paired policy-environment units. Environment repetitions are nested within policy seeds. Methods within an experimental group share demonstrations, RGB policy observations, task protocols, success criteria, and denominators. Geometric comparators share calibration and auxiliary-depth conditions. Section 4.1.3 defines aggregation; Tables A.3 and C.2 delimit the available budget records.

The comparisons include supervised imitation (Behavior Cloning, ACT, and Diffusion Policy), video or trajectory references (Track2Act-style Video Imitation, the diffusion/video trajectory baseline, and Hand-crafted Waypoint), and system variants (VLA without a 3DGS twin, Isaac Sim random augmentation, Naive Dual-arm VLA, and Structured Dual-arm VLA). Their information sources and module settings follow the definitions in each experiment.

4.1.2. Training implementation and data format

Training samples are organized into image, language, state, and action fields. “GR00T/LeRobot-style” describes only the data interface and does not identify the actual policy architecture, pretrained weights, or checkpoint. Each demonstration episode contains synchronized RGB images, optional depth or point clouds, robot states, actions, language annotations, task-stage labels, and a success flag; Table A.3 lists the training configurations. For the SO101 bimanual tasks, policy actions are recorded in the fixed order of 12-dimensional absolute joint position targets in Table A.4, with 5 actuated joints and 1 gripper component per arm. End-effector poses and relative bimanual

geometry are used only for relation supervision, local mapping, or trajectory diagnostics, and are not policy-action fields. Statistical normalization follows each backbone configuration within the policy, and outputs are denormalized into the robot command space before execution.

Table A.3 links the available temporal and optimizer settings to the result rows and separates the three VLA variants. GR00T-style VLA is a configuration reference, not an additional evaluated model. Action-chunk length, executed steps per call, and replanning frequency are distinct quantities; the compared policies use different action windows.

4.1.3. Evaluation metrics

Task success is measured per complete episode; recovery and compensation success are measured per matched trigger segment. Trajectory metrics use aligned samples, and safety metrics use the exposures in Table 7. Tables report means and sample standard deviations over paired policy–environment units. The analysis is descriptive: environment repetitions do not count as independent training runs. Tables B.1 and B.2 define the aggregation and distinguish stage-level from complete-episode rates.

Task success rate uses held-out real evaluation episodes as its statistical unit. It is defined as the number of successfully completed episodes divided by the total number of evaluation episodes.

$$\text{SR} = \frac{N_{\text{succ}}}{N_{\text{eval}}} \quad (32)$$

Trajectory error measures the mean deviation between executed and reference states, summed over aligned control steps within the same trigger segment or evaluation episode,

$$E_{\text{traj}} = \frac{1}{T} \sum_{t=1}^T \|x_t - x_t^{\text{ref}}\|_2 \quad (33)$$

Relative bimanual error measures whether the relative geometry of the left and right end effectors remains stable,

$$E_{\text{rel}} = \frac{1}{T} \sum_{t=1}^T \|(p_t^{\text{L}} - p_t^{\text{R}}) - (p_t^{\text{L,ref}} - p_t^{\text{R,ref}})\|_2 \quad (34)$$

Control smoothness is defined as the mean second-order position difference over consecutive three-frame windows and is used only for aligned-trajectory diagnostics in pouring and board wiping.

$$E_{\text{smooth}} = \frac{1}{T-2} \sum_{t=2}^{T-1} \|p_{t+1} - 2p_t + p_{t-1}\|_2 \quad (35)$$

Failed-step recovery rate measures the proportion of trigger segments that recover successfully, while compensation success rate measures the proportion that achieve the local goal after compensation. They are defined as follows.

$$\mathcal{R}_{\text{rec}} = \frac{N_{\text{rec}}}{N_{\text{fail}}}, \quad \mathcal{R}_{\text{comp}} = \frac{N_{\text{comp,succ}}}{N_{\text{comp}}} \quad (36)$$

Eq. (37) defines dimensionless ratios by event type. The first four entries in Table 7 are multiplied by 100 and reported as percentages, whereas the QP entry is multiplied by 100 and reported as “corrections per 100 accepted control steps.” The latter describes correction frequency among accepted steps and excludes the complete set of rejected-candidate attempts. Events may co-occur within an episode, so their rates cannot be added to obtain an overall risk probability. A metric is undefined when its denominator is zero.

$$\mathcal{E}_k = \frac{N_k}{N_{\text{den},k}}, \quad k \in \{\text{env}, \text{self}, \text{joint}, \text{rej}, \text{QP}\} \quad (37)$$

4.2. Base bimanual task execution with sparse demonstrations

We first evaluate policies trained on the same sparse demonstrations using the six held-out bimanual tasks in Fig. 5, aggregating success rates across four stage categories: approach–grasp, grasp–transport, target placement, and simple interaction. BC, ACT, Diffusion Policy, Track2Act-style imitation, and three VLA configurations are compared for base execution. Full Pipeline additionally includes physical-proxy expansion and deployment-stage compensation as a system-level reference.

The three VLA rows in Table 1 use separate arm outputs, concatenated joint outputs, and joint outputs with shared context and relation supervision, respectively. Table A.3 lists these functional differences. Full Pipeline also includes proxy expansion and deployment compensation.

Appendix Fig. A.2 provides representative real-video sequences; the stage-level comparisons are reported in Table 1.

To prevent performance in one stage from obscuring the others, Table 1 reports success rates separately for the four task-stage categories and provides their equally weighted macro-average in the final column.

Table 1: Success rates (%) for four task-stage categories with sparse demonstrations

Method	Approach and grasp ↑	Grasp and transport ↑	Target placement ↑	Simple in- teraction ↑	Stage macro- average ↑
Behavior Cloning (Trans- former)	51.33±3.03	47.33±4.02	48.44±3.75	53.56±4.27	50.17±1.45
ACT	59.11±3.44	52.89±2.78	54.89±2.48	59.33±3.61	56.56±1.78
Diffusion Policy	63.33±3.33	57.78±3.00	59.56±3.75	63.78±3.53	61.11±1.88
Track2Act-style Video Imitation	65.11±2.48	58.22±2.13	60.22±3.44	65.78±2.95	62.33±1.38
Single-arm VLA	65.11±3.53	57.56±2.66	60.00±1.78	64.89±2.48	61.89±1.24
Naive Dual-arm VLA	70.00±3.56	66.67±3.09	67.56±1.98	71.11±2.72	68.83±1.47
Structured Dual-arm VLA	78.45±2.47	72.22±2.72	73.33±2.18	79.33±2.58	75.83±0.83
Ours Full Pipeline†	91.33±2.11	86.00±1.87	87.56±1.52	91.11±2.72	89.00±0.72

Note: Mean $\pm$ sample SD across 15 paired units (5 policy seeds, 3 environment seeds each). Macro-averages weight the four stages equally within each unit; they are not complete-episode rates. Stage eligibility and denominator limits are detailed in Appendix B. † Includes proxy expansion and deployment compensation.

The full-system values in Tables 1, 4, 6, and 8 use different metrics or evaluation sets. Table 1 reports stage macro-averages; Table 4 uses complete episodes and matched recovery windows; Table 6 uses the ablation task pool; and Table 8 uses nominal and perturbed subsets. The Residual RL row in Table 2 is a separate extension.

Structured Dual-arm VLA has a stage macro-average of 75.83%, compared with 68.83% for Naive Dual-arm VLA, and a higher mean than ACT in each stage. Full Pipeline reaches 89.00%, with stage means from 86.0% to 91.3%. These are comparisons of the listed configurations, including the extra expansion and compensation used by Full Pipeline.

Approach–grasp, grasp–transport, target placement, and simple interaction are functional stages, not task identities. A task can involve several stages, so the stage ranking does not determine the end-to-end ranking of the six tasks.

The reported aggregates do not resolve task-to-stage eligibility or the treatment of stages unreached after an earlier failure. Table A.2 specifies

exclusion for missing keyframes; this rule does not determine the handling of unreached stages. Appendix B separates the known averaging procedure from the unavailable denominator accounting. Table 1 retains the reported values without reconstructing counts.

4.3. *Structured virtual-environment reconstruction and task-boundary expansion*

Table 2 compares the expansion procedures using real-robot task success, and Table 3 resolves the outcomes by stage. All virtual feedback follows the sampling and acceptance definitions in Table A.7.

To assess task-boundary expansion, Table 2 compares seven training settings: Sparse Demo uses only the original sparse demonstrations; the no-virtual-twin path only replays real trajectories; Isaac Sim Random Aug. uses uniform randomization; Uniform Virtual Rollout samples uniformly in the virtual environment and filters for successful trajectories; Video Trajectory Aug. replays generated references; Failure-region Expansion samples around failure regions and filters candidates; and Failure-region + Residual RL additionally refines candidates with a residual policy.

Table 2 compares complete expansion paths. The no-virtual-twin path uses only real-trajectory replay and removes both the complete virtual twin and policy-rollout branch. It therefore cannot be interpreted as a single-factor ablation of 3DGS versus TSDF geometry. Isaac Sim random augmentation and uniform virtual rollout provide stronger controls, while the failure-region setting evaluates the combined contribution of the object-level physical proxy, targeted sampling, and filtering.

Failure-region expansion reaches 84.33% real-robot success, compared with 78.83% for uniform virtual rollout. Simulation-only residual refinement reaches 86.00%. This ordering concerns the complete configurations; the effects of accepted data volume and computation are discussed in Section 5.

To further assess whether the method expands the executable task boundary, Table 3 reports changes in stage-specific failure rates before and after expansion.

In Table 3, failure-region expansion yields lower failure rates than uniform rollout for all four stage categories. Approach–grasp, grasp–transport, target placement, and simple interaction decrease from 12.67%, 17.47%, 20.53%, and 16.40% to 7.73%, 10.67%, 12.40%, and 8.80%, respectively. Target placement retains the highest mean failure rate after expansion, indicating that precise end-effector–target alignment remains a weakness in the current tasks.

Table 2: Success rates for different task-boundary expansion methods

Training setting	Expansion strategy	Failure-region guidance	Successful-trajectory filtering	Simulation success (%) ↑	Real-robot success (%) ↑
Sparse Demo	None	×	×	75.47 ± 2.20	73.00 ± 3.16
No virtual-twin path	Real-trajectory replay only	×	✓	77.60 ± 2.41	74.17 ± 3.49
Isaac Sim Random Aug.	Uniform randomization	×	✓	80.67 ± 2.79	75.67 ± 3.47
Uniform Virtual Rollout	Uniform state sampling	×	✓	84.00 ± 2.62	78.83 ± 2.81
Video Trajectory Aug.	Generated reference replay	×	✓	81.73 ± 2.12	78.67 ± 1.86
Failure-region Expansion	Boundary-guided sampling	✓	✓	89.20 ± 1.97	84.33 ± 2.58
Failure-region + Residual RL	Boundary-guided + residual RL	✓	✓	90.27 ± 2.37	86.00 ± 2.80

Note: Mean $\pm$ standard deviation; ✓ indicates enabled and × indicates disabled. Residual RL proposes candidates only in simulation. Candidates undergo the same acceptance procedure before training feedback; Residual RL is not used for deployment control.

Table 3: Stage-specific failure rates before and after task-boundary expansion

Task stage	Before expansion (%) ↓	After uniform rollout (%) ↓	After failure-region expansion (%) ↓
Approach-Grasp	17.60 ± 1.55	12.67 ± 1.63	7.73 ± 1.03
Pick-and-Transport	24.27 ± 1.67	17.47 ± 2.20	10.67 ± 1.63
Place-to-Target	27.07 ± 2.60	20.53 ± 1.92	12.40 ± 1.88
Simple Interaction	22.27 ± 2.12	16.40 ± 1.88	8.80 ± 2.24

Note: Values are mean $\pm$ standard deviation of stage failure rates, using evaluation episode–stage pairs as the statistical unit.

Fig. 6 illustrates the object-level modeling inputs used by the expansion procedure. Objects are separated from the background, and articulated parts receive joint axes and ranges. Fig. A.1 shows corresponding real and proxy scenes; construction settings are listed in Table A.6.

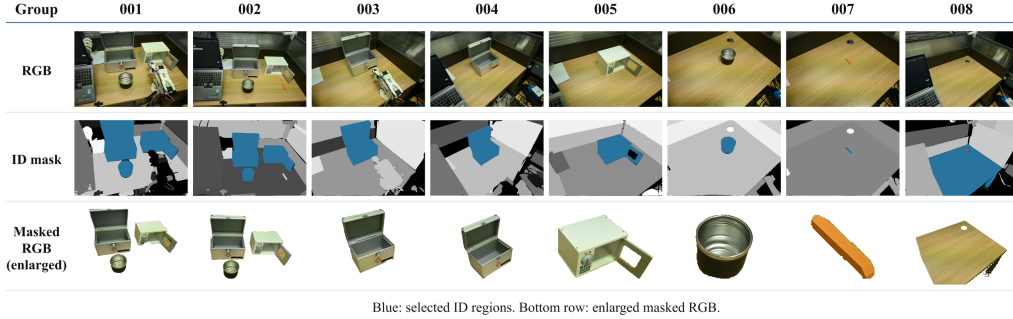


Figure 6: RGB observations, ID masks, and region extraction for object-level modeling. Columns correspond to acquisition groups 001–008. The first two rows show same-view RGB images and ID masks, with selected regions marked in blue; the third row shows enlarged RGB content within the masks.

Fig. 6 contains eight acquisition groups from one tabletop environment. Groups 001–002 select the medicine box, microwave, and pot together; 003–007 show individual regions; 008 shows the tabletop. IDs are local to each image. Occlusions and segmentation errors remain visible, making this an illustration of modeling inputs rather than a reconstruction-accuracy test.

4.4. Trajectory-generation compensation for failed steps on the real robot

Table 4 evaluates local recovery without retraining the policy. The comparisons separate base-policy continuation, retries, manually specified waypoints, video references, and the full compensation procedure.

The full procedure uses the asynchronous mapping and risk gate in Section 3.3; the base VLA continues through the shared filter while a reference is pending or invalid.

Table 4 compares five real-robot execution settings. Base Policy uses only the policy augmented in Section 4.3, without failure compensation. Policy Retry repeats the base policy after failure. Hand-crafted Waypoint uses manually specified local intermediate points. Diffusion/Video Trajectory Baseline uses generated-video references. Ours Trajectory Compensation extracts 2-D trajectories from generated videos, obtains local reference actions

through current-state alignment and task-masked DLS inverse kinematics, converts them into the same 12-dimensional absolute joint position target space as the base VLA, and applies gated fusion and QP feasibility projection. Step recovery and compensation success are computed over matched trigger windows, while task success and collision rates use the corresponding complete evaluation episodes.

Table 4: Compensation performance for failed steps on the real robot

Execution method	Compensation mechanism	Step recovery (%) $\uparrow$	Task success (%) $\uparrow$	Collision (%) $\downarrow$	Compensation success (%) $\uparrow$
Base Policy	$\times$	1.33 ± 2.11	72.17 ± 2.65	8.50 ± 1.27	N/A
Policy Retry	Repeat base policy	30.00 ± 3.56	75.17 ± 2.40	9.83 ± 1.14	22.00 ± 3.52
Hand-crafted Waypoint	Manual local waypoint	42.44 ± 2.95	78.50 ± 2.80	7.50 ± 1.34	36.45 ± 3.21
Diffusion/Video Trajectory Baseline	Generated video reference	53.33 ± 2.18	80.50 ± 2.15	8.67 ± 1.29	45.78 ± 3.88
Ours Trajectory Compensation	Alignment and action mapping; gated fusion; shared safety filter	67.78 ± 3.00	85.33 ± 2.29	5.50 ± 1.04	64.22 ± 3.20

Note: Mean $\pm$ standard deviation. Step recovery and compensation success use matched trigger segments; task success and collision use complete episodes. N/A indicates that compensation is not enabled for the base policy.

In Table 4, step recovery and compensation success use matched trigger segments, while complete-task success and collision use the corresponding evaluation episodes. Waiting, reference rejection, or non-adoption is not automatically counted as compensation success, and the original matched-segment denominator cannot be replaced by the number of segments that actually adopted a reference. Compensation is undefined for Base Policy and is therefore N/A. The reference rejection rate in Table 7 is not the complement of the adoption rate: even a reference that passes validity checks may not be adopted if current risk decreases or the task ends.

In Table 4, full compensation achieves a step recovery rate of 67.78% on matched trigger segments, compared with 53.33%, 42.44%, 30.00%, and

1.33% for the video-trajectory, hand-crafted-waypoint, retry, and base-policy settings, respectively. The corresponding complete evaluation episodes have a task success rate of 85.33% and a collision rate of 5.50%; collision rates for the four controls range from 7.50% to 9.83%. Providing a reference alone is associated with some recovery, while the full configuration combines current-state mapping, risk gating, and shared filtering and yields a higher mean recovery rate and a lower mean collision rate.

Table A.8 defines reference acceptance and fallback. Offline trajectory diagnostics are reported separately in Figs. A.3–A.4.

To further examine trajectory-generation compensation across failure types, Table 5 reports recovery results for typical failure scenarios.

Table 5: Trajectory-generation compensation across failure types

Failure type	Main stage	Base-policy local success rate (%) $\uparrow$	Local success rate after compensation (%) $\uparrow$
Grasp Offset	Approach-Grasp	61.07 $\pm$ 2.81	80.80 $\pm$ 2.24
Object Slip	Pick-and-Transport	58.67 $\pm$ 3.27	76.80 $\pm$ 3.45
Pose Misalignment	Place-to-Target	52.53 $\pm$ 4.24	77.87 $\pm$ 3.66
Unseen Object State	Simple Interaction	48.53 $\pm$ 4.24	71.47 $\pm$ 3.34

Note: Mean $\pm$ standard deviation. Both columns use matched segments and the same denominator within each failure type and report local-segment success rates.

Supplementary Videos S1 and S2 provide retained real-robot clips of pouring-related manipulation and board wiping, respectively, corresponding to the qualitative interactions in Fig. 7. They illustrate local motion and interaction geometry; they are not synchronized records of risk triggering, reference adoption, or online recovery.

Table 5 compares matched segments within each failure type, using the same denominator for both columns. These are local recovery outcomes, distinct from complete-episode success.

In Fig. 7, marker 1 in the pouring row identifies the visible body of the held container at 1.60 s; marker 2 identifies the receiving container at 1.60 s and 2.83 s. The occluded container mouth is outside the annotation. Marker 3 in the wiping row identifies the eraser and adjacent board surface at 1.57

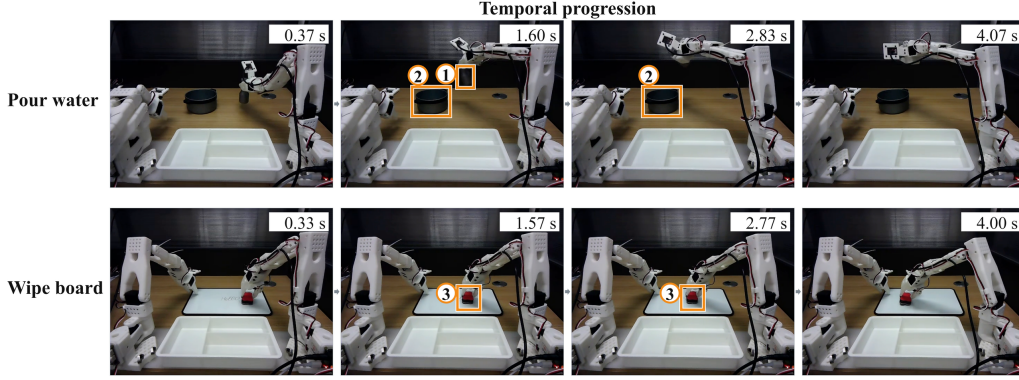


Figure 7: Real-robot interaction sequences for pouring and board wiping. Time advances from left to right; the upper-right label in each frame gives the actual time (s) within its source clip, with the two rows timed independently. Orange boxes with white outlines and numbered labels mark interaction regions of interest: 1 denotes the visible part of the held container, 2 the receiving container, and 3 the eraser and adjacent board surface. Numbers link the figure to the text and do not denote risk triggers, compensation activation, or successful recovery.

s and 2.77 s. The figure explains interaction locations for the two tasks; Tables 4 and 5 quantify compensation performance. Offline image trajectories and numerical diagnostics are shown in Figs. A.3 and A.4 and are interpreted separately from the online action fusion in Section 3.3.

4.5. Ablation studies

We compare system-level results after removing or replacing key modules. The five components are structured bimanual VLA, the object-level virtual environment, failure-region-guided expansion, successful-trajectory filtering, and trajectory compensation. Table 6 reports descriptive results for each configuration on a matched task pool; the module indicators in each row define its composition.

The ablation settings are as follows. w/o Structured Dual-arm replaces the structured policy with a simple concatenated bimanual-action policy. w/o Structured Virtual Env. uses a standard scene-level virtual environment. w/o Boundary Expansion switches to uniform sampling. w/o Success Filtering adds virtual trajectories without filtering by successful outcomes. w/o Trajectory Compensation disables compensation in both simulation and real-robot evaluation; the base policy continues after local failures, or outcomes are

determined by the protocol. Table 6 specifies the retained modules in each row.

Table 6 compares overall performance across the ablation settings.

Full Model has the highest mean simulation success, real-robot success, and step recovery in Table 6. Removing the structured policy or object-level proxy gives 74.00% real-robot success, compared with 86.17% for Full Model. Removing compensation lowers recovery from 69.56% to 31.78%, the largest recovery drop among the listed variants. These results compare complete configurations; data and computation are considered in Section 5.

The standard scene-level environment in the proxy ablation does not use the same object-level decomposition. Its 74.00% real-robot success is a configuration-level outcome; the experiment does not measure which geometry or contact error accounts for the difference.

w/o Boundary Expansion replaces failure-region sampling with uniform sampling, yielding 77.50% real-robot success compared with 86.17% for the full configuration. This is consistent with the sampling-configuration ranking in Table 2. Section 5 discusses comparability limits for candidate counts, acceptance counts, and training budgets.

Without successful-trajectory filtering, real-robot success is 77.83% and collision is 9.17%, compared with 86.17% and 5.17% for Full Model. This comparison includes the changed composition of returned training data.

Table 6: Ablation results for key modules

Model configura- tion	SV	PP	FE	SF	SC	Simulation success (%) ↑	Real-robot success (%) ↑	Step recov- ery (%) ↑	Collision (%) ↓
w/o Structured Dual- arm	×	✓	✓	✓	✓	81.47 ± 2.07	74.00 ± 3.11	56.44 ± 3.44	8.17 ± 1.14
w/o Structured Vir- tual Env.	✓	×	✓	✓	✓	81.87 ± 1.41	74.00 ± 2.46	57.56 ± 4.27	7.17 ± 1.29
w/o Boundary Ex- pansion	✓	✓	×	✓	✓	83.73 ± 2.37	77.50 ± 1.64	58.89 ± 3.92	7.33 ± 1.76
w/o Success Filtering	✓	✓	✓	×	✓	84.80 ± 1.47	77.83 ± 2.29	59.55 ± 4.69	9.17 ± 1.22
w/o Trajectory Com- pensation	✓	✓	✓	✓	×	88.80 ± 2.11	78.50 ± 2.07	31.78 ± 3.96	5.67 ± 1.14
Full Model	✓	✓	✓	✓	✓	90.67 ± 1.95	86.17 ± 2.29	69.56 ± 4.86	5.17 ± 1.48

Note: Mean $\pm$ standard deviation. ✓ indicates enabled; × indicates removal or replacement, as detailed in the text. SV: structured bimanual VLA; PP: object-level physical proxy; FE: failure-region expansion; SF: successful-trajectory filtering; SC: step-level compensation.

4.6. Safety and failure-risk analysis

Table 7 separately reports environmental collision, self-collision, joint-structure violations, generated-reference rejection, and QP corrections, complementing the local recovery results in Table 4. References undergo trajectory, current-state, and reachability checks before action fusion; fused commands then pass through the shared QP and nonlinear recheck. Failure to meet the relevant conditions causes reference rejection or execution fallback.

Environmental collisions, self-collisions, and joint-structure violations are normalized by held-out real-robot evaluation episodes; reference rejection is normalized by generated references; and QP corrections are counted per 100 accepted control steps. These distinct exposure units describe episode-level events, reference availability, and control-intervention frequency. Section 5 consolidates the scope of the safety conclusions.

The 2.20% joint-structure event rate in Table 7 combines robot and object joint-range events. The detection point in the candidate-filter-execution chain is unspecified, so this quantity cannot be interpreted as a post-filter violation probability.

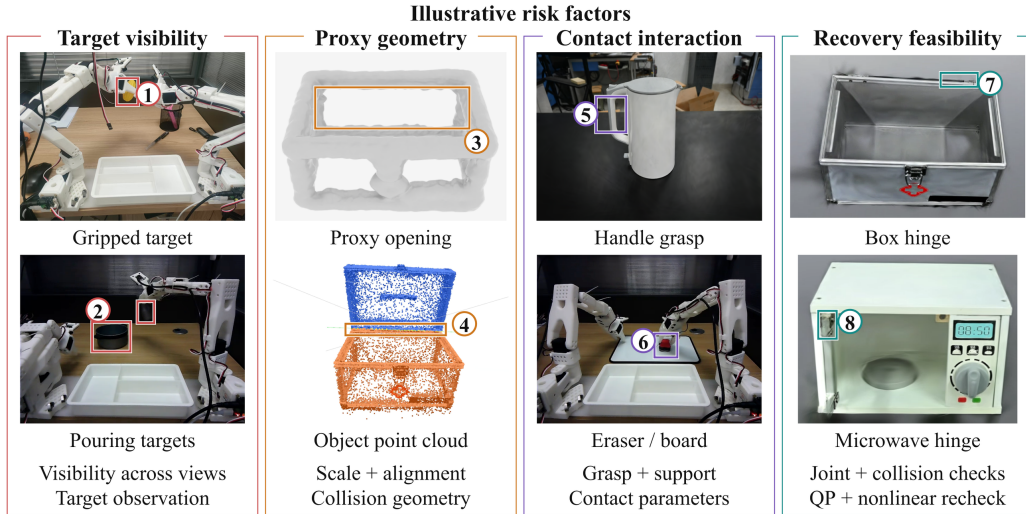


Figure 8: Illustrative potential failure factors and inspection points. Two examples appear in each column: 1–2 show holding and pouring targets, 3–4 a proxy opening and object point cloud, 5–6 a cup handle and eraser, and 7–8 a box lid and microwave hinge. Markers indicate observation regions only, not annotated failure episodes.

Table 7: Safety events and QP correction frequency of the full system

Safety event	Definition	Statistical unit and denominator	Reported value
Environmental collision	Robot, object, or tool violates environmental clearance	Collision episodes / total evaluation episodes	$4.53 \pm 0.74\%$
Self-collision	Robot link or end effector violates self-collision clearance	Self-collision episodes / total evaluation episodes	$1.47 \pm 0.64\%$
Joint-structure violation	Command exceeds a robot or object joint range	Limit violations / total evaluation episodes	$2.20 \pm 0.77\%$
Generated-reference rejection	Reference fails trajectory extraction, state alignment, mapping, or acceptance checks and does not enter action fusion	Rejected references / generated references	$9.13 \pm 1.96\%$
QP safety correction	Safety QP modifies a fused action to satisfy constraints	Corrections per 100 accepted control steps	7.13 ± 2.03 corrections

Note: Mean $\pm$ standard deviation. The first four entries are percentages; QP corrections are reported per 100 accepted control steps. Each entry is interpreted using its own denominator.

Fig. 8 maps eight examples to checks of visibility, geometry, contact, and recovery feasibility, helping locate container mouths, tools, and articulated regions that require inspection. It illustrates inspection locations; Table 7 reports event frequencies, while Tables A.6 and A.8 specify proxy configurations and reference rejection rules, respectively.

4.7. Robustness and statistical analysis

Table 8 reports lighting, depth noise, camera pose, clutter, friction, and unseen-object-state conditions. “OOD Object State” names the held-out state group, not new object identities or new tasks. Perturbation magnitudes are unavailable, so comparisons apply to these particular settings.

The nominal condition has 85.71% task success and 4.76% collision. Each perturbation condition has a lower success mean and a higher collision mean. OOD Object State shows the largest change in this set, reaching 77.52% success and 6.85% collision; Lighting Shift retains 83.24% success. Because

perturbation strengths are not matched on a common scale, these differences do not rank intrinsic sensitivity to the six factors.

Table 8: Success and collision rates under stress tests

Test condition	Task success (%) $\uparrow$	Collision (%) $\downarrow$
Camera Pose Offset	80.19 ± 4.11	6.09 ± 1.48
Cluttered Tabletop	81.14 ± 3.20	6.09 ± 1.48
Depth Noise	81.71 ± 3.71	5.33 ± 1.00
Friction Shift	80.19 ± 3.66	6.66 ± 1.77
Lighting Shift	83.24 ± 2.61	5.71 ± 1.87
Nominal	85.71 ± 3.05	4.76 ± 1.76
OOD Object State	77.52 ± 3.03	6.85 ± 1.45

Note: Mean $\pm$ standard deviation. Nominal denotes the nominal condition; other rows apply single-factor perturbations.

5. Discussion and limitations

The comparisons point to two complementary uses of additional experience. Training-time expansion improves execution beyond the original demonstration neighborhoods, while deployment compensation addresses local failures after training. The large recovery drop when compensation is removed in Table 6 is consistent with this division of roles. The structured VLA results concern the combined interface and losses; the experiment does not isolate contact, role, relation-loss, or safety-prior effects.

Three reporting gaps limit attribution. First, per-task demonstration counts, task-to-stage assignments, stage denominators, and exclusion counts are unavailable, so Table 1 supports stage-level descriptive comparisons rather than a sample-efficiency claim. Second, candidate counts, accepted data volume, and optimization budgets are not jointly recorded for Tables 2 and 6; additional data or computation may account for part of the gains. Third, per-round expansion records, per-object proxy errors and admission thresholds, and online risk-gate-recovery traces are absent. The endpoint tables cannot establish these process-level properties.

Generated references take approximately two minutes and are useful only when the task permits delayed guidance and the returned trajectory remains

alignable. Occlusion, calibration drift, slip, or changed contact can invalidate it. Reference expiry, adoption rate, waiting-time accounting, and matched recovery budgets are unavailable, so Table 4 does not establish recovery efficiency at equal time cost. Table A.9 separates the known generator identity from the incomplete online tracking configuration.

Tables 1–8 provide the quantitative comparisons. Figs. 6–8 illustrate acquisition, interaction, and inspection locations; Figs. A.1–A.2 add visual examples. The offline processing in Figs. A.3–A.4 is not a measurement of online compensation. Table 7 describes observed events and interventions, not a safety certification. Table C.2 summarizes the scope of the evidence.

6. Conclusion

More from Less combines structured bimanual action prediction, failure-region expansion with physical proxies, and gated local recovery. In the reported six-task evaluation, Full Pipeline reaches a four-stage macro-average of 89.00%, compared with 75.83% for Structured Dual-arm VLA. Expansion and compensation configurations also have higher success and recovery means in their respective comparisons. The evidence supports these system-level differences; demonstration-count scaling, equal-budget module effects, and online recovery timing remain unresolved.

Declarations

Funding. Details withheld for anonymous review.

Competing interests. The authors declare no relevant financial or non-financial competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data and code availability. The project page (<https://robovla.github.io/More-from-Less/>) provides the manuscript, reported aggregates, selected media, retained reconstruction/simulation utilities, and saved-trajectory reproduction code. New reference implementations cover structured training, failure-guided expansion, and shared-filter control. Tested on analytic fixtures, they are not the historical implementations that produced the reported results. Original checkpoints, demonstration splits, and detailed run logs are not included; this release does not fully reproduce Tables 1–8.

CRedit authorship contribution statement. Author-specific contributions are withheld for anonymous review.

Appendix A. Experimental protocol and implementation details

This appendix summarizes the recorded data interfaces, parameter settings, and evaluation definitions. Tables A.1–A.8 cover data, policies, actions, proxy expansion, and deployment recovery; Table A.9 adds verifiable video-generation and offline-tracking configurations.

Table A.1: Dataset scope and task suite

Component	Contents	Split unit	Use in this study
Real demonstrations	Synchronized RGB images, multiview videos, robot states, language, and step annotations; depth recorded as non-policy geometric support	Evaluation-episode-level split	RGB base-policy training, physical-proxy construction, and held-out evaluation
Bimanual tasks	Bimanual water pouring, microwave opening, medicine-box opening, board wiping, wipe-and-pour composite task, and hold-and-wipe-board cooperative task	Retain task and object identities	Base execution, recovery, and robustness evaluation
Object-level scenes	RGB images, masks, PLY files, and sparse-reconstruction files for scenes 001–009	Scene identity follows the source-object split	3DGS reconstruction and physical-proxy construction
Virtual trajectories	Isaac Sim policy roll-outs initiated from failure states in the training or adaptation set	Inherit source split	Failure-region expansion and successful-trajectory feedback
Compensation segments	Matched high-risk local windows, including accepted and rejected references	Held-out trigger windows	Step recovery and collision analysis

Table A.2: Data separation, synchronization, and preprocessing

Item	Protocol	Processing	Purpose
Data separation	Keep all modalities from the same execution within one split	Hold out object and composite-task identities when required	Prevent view, evaluation-episode, and object leakage
Video format	Most streams are 640 × 480 at 30 Hz; retain actual metadata for each stream	Resample only to the policy or analysis frequency	Avoid implicit mixing of frame rates and resolutions
Temporal alignment	Evaluation-episode clock and nearest-neighbor matching	Interpolate short gaps in low-dimensional states; mark RGB and depth as unavailable	Do not synthesize missing visual evidence
Calibration	Camera intrinsics, hand-eye or table-top/base extrinsics, and depth scale	Fix transforms for each evaluation batch	Provide metric 3-D geometry and a common coordinate frame
Visual preprocessing	Resizing, cropping, color normalization, valid-depth masks, and object/part masks	Use consistent settings within each comparison	Control differences in sensor information
Action preprocessing	Radians or metric units and a fixed left/right-arm field order	Normalize actions and robot states using training-set statistics	Keep policy, replay, and control interfaces consistent
Missing key-stage frames	Do not compute the stage metric when key contact, pouring, or recovery frames are missing	Retain the evaluation episode for unaffected analyses	Avoid inventing evidence for missing events
Unreached / inapplicable stages	Distinct from missing keyframes	Original inclusion rules and separate counts are unavailable	Limits the denominator audit of Table 1; see Appendix B

Table A.3: Recorded policy configurations, VLA variants, and corresponding results

Policy / variant	Temporal / action interface	Optimizer / supervision	Result mapping and status
ACT	1 observation step, chunk 100, 100 action steps	AdamW; learning rate 1e-5, weight decay 1e-4, backbone learning rate 1e-5	Table 1: ACT
Diffusion Policy	2 observation steps, horizon 16, 8 action steps; discard the last 7 frames	Adam; learning rate 1e-4, betas (0.95, 0.999), epsilon 1e-8	Table 1: Diffusion Policy
Dual-arm VLA (re-reported configuration)	1 observation step, chunk 32, 32 action steps	AdamW; learning rate 1e-4, betas (0.9, 0.99), gradient clipping 10	Interface configuration for the bimanual VLA family in Table 1; see the variant rows below
GR00T-style VLA	1 observation step, chunk 50, 50 action steps	AdamW; learning rate 1e-4, batch size 32, 10,000 training steps	Configuration reference; no uniquely corresponding independent result row
BC or Transformer BC	Matching observation and action windows is a design principle; actual windows are not specified	Select checkpoints using validation data; matched effective batch size and update budgets remain unverified	Table 1: Behavior Cloning (Transformer)
<i>VLA variants and functional differences</i>			
Single-arm VLA	Separate action outputs for the left and right arms	Independent-control baseline without the proposed cross-arm relation supervision	Table 1: Single-arm VLA
Naive Dual-arm VLA	Joint output of concatenated actions for both arms	Without the proposed relation loss	Table 1: Naive Dual-arm VLA; replacement when the structured policy is removed in Table 6
Structured Dual-arm VLA	Shared task context; joint 32×12 absolute joint position targets	Adds training-only relation supervision; relation variables do not enter forward inference	Table 1: Structured Dual-arm VLA
Full Pipeline / Full Model	Structured bimanual policy	Additionally includes proxy expansion, success filtering, and deployment compensation; module switches follow each table	System reference in Table 1; full configuration in Table 6; evaluated on different populations

Configuration scope: the Dual-arm VLA settings are not bound individually to every variant. Backbone checkpoints, parameter counts, complete update budgets, and per-baseline safety-filter settings are therefore not established as identical. GR00T-style is a reference configuration only. For Eq. (7), numerical group weights, normalization scales, and the two overall loss weights are unavailable; an action-conditioned contact or role loss is not specified. The structured result row evaluates the combined configuration.

Table A.4: SO101 bimanual action interface

Dimensions	Side	Field order	Shared use
0–5	Left arm	left_shoulder_pan, left_shoulder_lift, left_elbow_flex, left_wrist_flex, left_wrist_roll, left_gripper	Policy output, replay, and control
6–11	Right arm	right_shoulder_pan, right_shoulder_lift, right_elbow_flex, right_wrist_flex, right_wrist_roll, right_gripper	Policy output, replay, and control
Single arm	Robot	shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper	SO101 order in the code
Trajectory	Either side	joint_names, time-varying joint positions, control interval, task, and outcome	Virtual training feedback and compensation replay
Normalization	Both arms	Training-set means and scales, applied consistently during training and deployment	Stable joint-action prediction

Table A.5: Checkpoint selection, IK, and local safety projection

Component	Definition	Selection or constraint	Fallback
Checkpoint	Select one checkpoint independently for each policy seed	Use validation-set action, trajectory, coordination, and safety metrics only	No test-set tuning
Damped least-squares IK	Map left/right end-effector references to joint increments	Use positive damping near singular configurations and check residuals	Reject unreachable references
QP weights	Positive-definite action-correction matrix and positive slack penalty	Penalize command modifications and bounded soft-constraint slack	Hold or slow down when slack is excessive
Command bounds	Absolute joint ranges and per-step change bounds relative to the current state and previous safe command	Apply in a common 12-dimensional absolute joint position target space	Reject out-of-bound candidates; recheck fallback commands
Safety Jacobian	One row per environmental, self-collision, structural, or contact margin	Recompute around the raw fused action each cycle	Reject when the convex subproblem is infeasible
Smoothing	Low-pass filtering of gate weights and joint commands	Before the QP; no command modification after rechecking	Suppress switching transients and action jitter

Table A.6: 3DGS physical-proxy construction and representative checks

Stage or check	Configuration	Function	Use in policy roll-outs
Gaussian filtering	Minimum opacity 0.005, cluster margin 0.12, cluster opacity 0.7, epsilon 0.5, minimum 10 points	Remove low-confidence and isolated primitives	Generate object-support points
Mesh extraction	Mesh opacity 0.16; simplification target 25,000 faces; conservative contact-surface inflation 2–5 mm	Construct visual and collision geometry	Reduce penetration-prone contacts
Scene transform	Position (1.2, -0.095, 0.06), rotation (35, -1, 5) degrees, scale 0.08	Align with the Isaac Sim workspace	Fixed in the reported scene
Movable structures	Revolute or prismatic joints with calibrated axes and motion ranges	Represent hinges, rails, and drawers	Constrain structurally feasible sampling
Geometry checks	Bounding-box error and Chamfer distance	Check metric scale and shape coverage	Exclude proxies with inconsistent contact geometry from policy rollouts
Structure and response checks	Joint-axis error, joint-range consistency, and short-horizon push-response error	Check structural directions and coarse contact responses	Representative proxy screening
Transfer checks	Short-horizon sim-to-real pose discrepancy	Diagnose proxy-induced transfer errors	No claim of comprehensive metrological validation across all objects

Table A.7: Failure-region sampling and virtual-trajectory feedback

Element	State variables	Predicate	Training role
Failure state	Task stage, object 6D pose, structural state, left/right end-effector poses, contact, uncertainty, and deviation	Current policy violates task or safety predicates	Define local expansion centers
Candidate state	Perturbations of object-frame poses, contact points, structural states, end-effector targets, and target regions	Reachability, support, joint ranges, and contact consistency	Exclude unconstrained random states
Current-policy rollout	The same current policy associated with the failure region being expanded	Satisfy task stages in sequence	Avoid an unspecified auxiliary action generator
Safety filtering	Environmental collision, self-collision, joint limits, and object penetration	Pass all safety predicates	Do not return unsafe trajectories to training
Physical consistency	Continuous object states and plausible contact responses	No state jumps, penetration, or abnormal motion	Reduce sim-to-real contamination
Mixed training	Real demonstrations and quality-weighted accepted virtual trajectories	Real-data mixture weight exceeds one half; the total coefficient of the quality-weighted virtual term remains below that of the real term	Virtual samples supplement only weak but feasible states

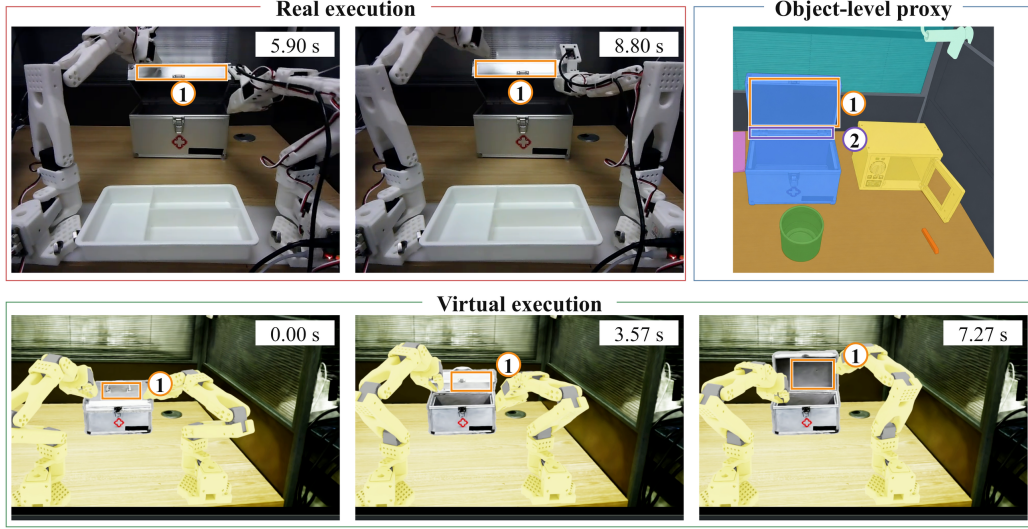


Figure A.1: Real execution, object-level proxy, and virtual execution for medicine-box opening. The upper row contains two real-robot frames and the proxy scene; the lower row contains three simulation frames. Marker 1 indicates the visible lid region and marker 2 the proxy articulation region. Times are measured independently in the respective videos; rows do not imply synchronized states.

Fig. A.1 shows visual correspondences among the real scene, object structure, and proxy execution. Candidate trajectories undergo joint acceptance for task, physics, and safety according to Table A.7; only successful complete virtual trajectories participate in real-dominant retraining.

Table A.8: Generated-video trajectory extraction, triggering, and fallback

Stage	Deterministic rule	Output	Rejection or applicability boundary
Target-region tracking	Track target objects, tools, or task-relevant regions in generated frames	Per-frame image coordinates, masks, and confidence	Reject low-confidence tracks, identity switches, or broken trajectories
Image-trajectory extraction	Connect valid coordinates, remove outliers, and apply temporal smoothing	Continuous 2-D video trajectory	Reject long gaps or discontinuous trajectories
Current-state alignment	Combine current observations, camera calibration, robot states, and task stage; depth supplies only current scale and obstacles	Displacements in a local task coordinate frame	Reject ambiguous alignment or workspace violations
Local action mapping	Construct task masks from the current grasp and roles; DLS solves 10 arm joints, while both grippers inherit VLA outputs	12-dimensional absolute joint position reference targets in a fixed order	Reject unreachable or relationally conflicting references
Hard trigger	Policy uncertainty, trajectory deviation, geometric risk, or stage stagnation exceeds its threshold and the reference is valid	Binary fusion enable	No fusion while the reference is pending, invalid, or unreachable
Action fusion	A bounded gate fuses base actions and accepted references in a common action space	Raw fused action	Generated video does not enter the VLA backbone
Runtime boundary	Video generation is asynchronous and takes approximately minutes; trajectory mapping, IK, and QP remain in the control chain	Low-frequency local recovery	While waiting for or rejecting a reference, $\alpha = 0$; base actions still pass through the shared filter. Slow/hold only if the QP fails or nonlinear rechecking rejects the candidate

Table A.9: Identities and configurations for video generation and offline trajectory processing

Component	Verified identity and configuration	Materials and applicability
Online generation	Kling AI 3.0; current RGB and language inputs; asynchronous call, approximately 2 min per generation	Generator version confirmed; mode, prompt template, seed, and service-internal checkpoint not fully retained
Saved videos	Pouring / board wiping: 1108×828, 30 fps, 72 / 75 frames, 2.40 / 2.50 s	Original MP4 metadata; saved-clip duration, not generation latency or the original service duration setting
2-D trajectory extraction from generated clips	OpenCV connected components and weighted centroids; motion-layer $\alpha > 24$, minimum area 200 px; code default: 5-frame mean smoothing	<code>moving_object_trajectory.json</code> for both tasks confirms the thresholds; the smoothing window is a code default not separately recorded in run metadata
Real pouring trajectory retargeting	OpenCV pyramidal LK optical flow; maximum 80 corners, quality threshold 0.01, minimum spacing 5 px; window 25×25, <code>maxLevel=3</code> , maximum 30 iterations, <code>epsilon=0.01</code> ; at least 8 valid points	Offline pouring-retargeting script and output metadata; offline inputs for Figs. A.3–A.4
Real board-wiping trajectory retargeting	OpenCV HSV red/dark eraser detection; $H < 13$ or $H > 168$, $S > 55$, $V > 35$; candidate area 30–9000 px, opening 3×3, closing 7×7	Offline board-wiping-retargeting script and output metadata; offline inputs for Figs. A.3–A.4

Offline extraction and retargeting use OpenCV operators, with color, frame-difference, and geometric rules for masks; pouring retargeting also interpolates missing points. These settings do not specify the online tracker in Table 4. Its build, confidence function, and thresholds are unavailable, as are the repeated-prediction count, risk weights, normalization scales, and gating thresholds in Eq. (26).

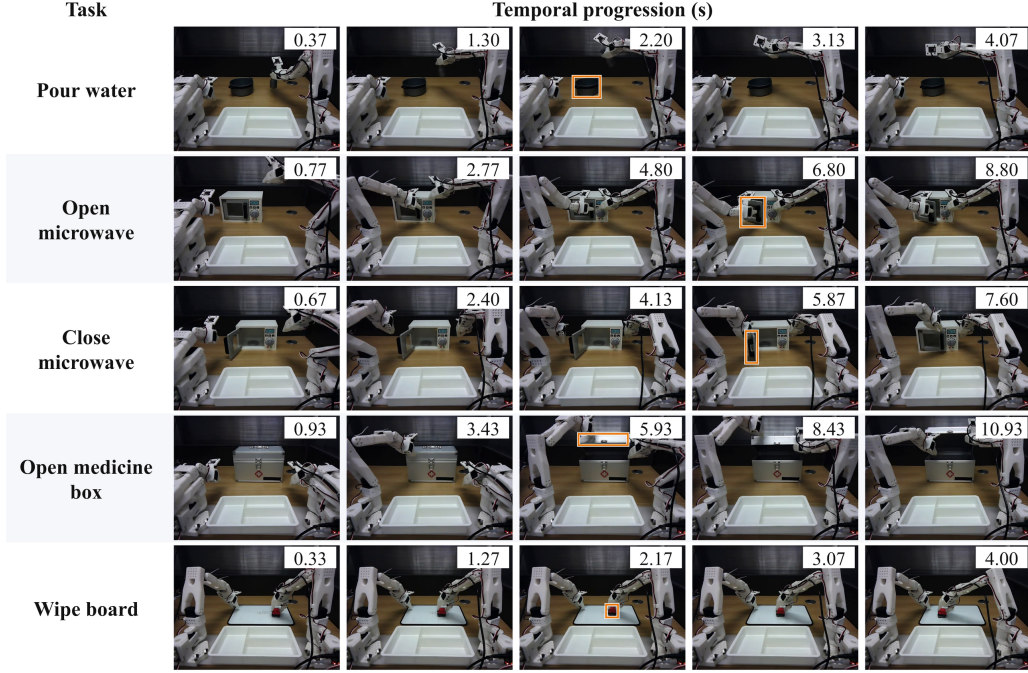
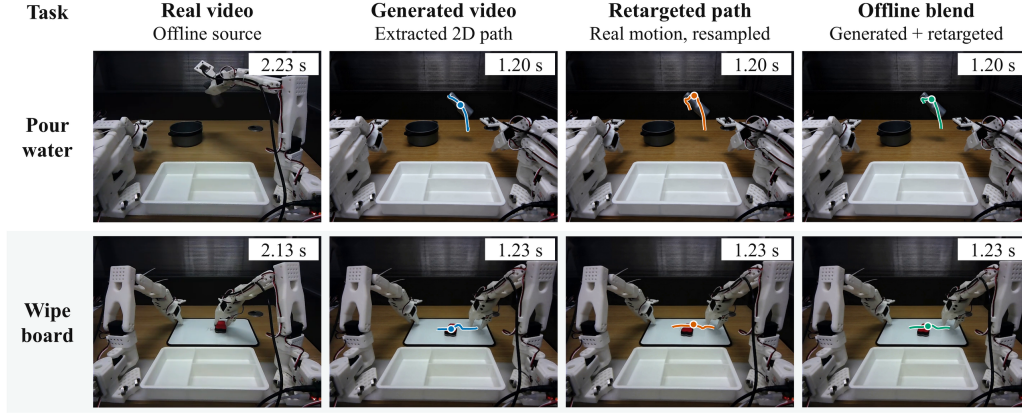


Figure A.2: Representative temporal frames from five groups of real bimanual action clips. From top to bottom, the rows show pouring, microwave opening, microwave closing, medicine-box opening, and board wiping. Each row comes from one video; the five columns follow temporal order, with source-video time (s) shown in the upper-right corner. Orange boxes mark interaction regions of interest, such as containers, doors, box lids, and erasers.

Fig. A.2 retains global views of both arms and the manipulated objects. Rows are timed independently, and the five columns do not constitute a common five-stage division. The board-wiping row covers only wiping execution. Orange boxes are manual observation cues; Table 1 gives quantitative stage results.



Full-clip paths; dots mark the displayed frame. Times are relative to each source clip.

Figure A.3: Offline image-trajectory diagnostics for pouring and board wiping. The four columns show real-video frames, trajectories extracted from generated videos, retargeted real-video trajectories, and offline fused trajectories, respectively. The last three columns overlay complete pixel trajectories on the same generated frame, with dots marking the trajectory positions corresponding to that frame. Upper-right labels give relative times in the respective source videos; resampling does not imply synchronization between real and generated videos.

Fig. A.3 compares existing pixel trajectories after resampling real motion to the generated-clip length. Pouring uses bounding-box range mapping, and wiping uses resolution scaling. The offline blend weights and smooths these trajectories; it does not use the online VLA action and risk gate. The associated 3-D reconstruction uses estimated intrinsics and synthetic motion-layer depth, restricting this example to offline trajectory inspection.

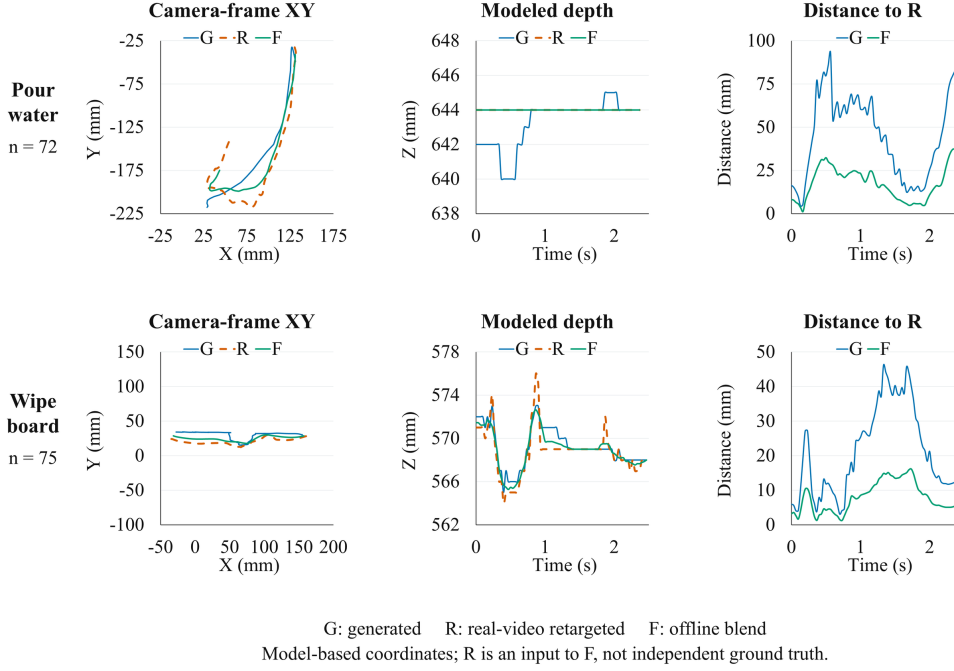


Figure A.4: Numerical diagnostics of offline pouring and board-wiping trajectories. Each row represents one clip, and n denotes the trajectory-point count. The three columns show the camera-frame XY path, modeled depth Z , and 3-D Euclidean distance to the retargeted trajectory R . G denotes the trajectory extracted from the generated video and mapped into 3-D; F denotes the offline fused trajectory. The XY axes have equal scales, and curves are not independently normalized. Time is the relative sampling time of the target generated clip.

The retargeted trajectory R is used to construct the offline blend F in Fig. A.4; a shorter F -to- R distance is therefore not an independent accuracy test. Coordinates depend on estimated intrinsics and synthetic motion-layer depth. For pouring, R and F both use a fixed Z of 644 mm. These curves describe that processing operation, not calibrated 6-D motion or online robot execution.

As an offline diagnostic, the mean distances from the fused trajectories to the retargeting inputs are 61.2% and 62.0% smaller than those from the generated trajectories in the pouring and board-wiping clips, respectively. Appendix Fig. A.4 provides the curves and calculation definitions. Because the retargeted trajectory itself participates in fusion, these differences are internal-consistency statistics, not evidence of independent accuracy or online

compensation performance.

At 30 Hz sampling, the pouring and board-wiping trajectories contain 72 and 75 points, spanning 2.37 and 2.47 s between the first and last samples. Under the existing camera-coordinate estimates, F has path lengths of 243.6 and 196.6 mm, respectively. The mean G/F distances to R are 44.2/17.2 mm for pouring and 20.9/7.9 mm for board wiping; maximum distances are 93.1/37.3 and 46.1/16.2 mm, respectively. The mean norms of second-order position differences over adjacent triplets are 6.93/0.73 mm for pouring and 4.01/0.51 mm for board wiping for G/F, respectively. These calculations are not divided by the squared sampling interval and cannot be interpreted as acceleration. These descriptive quantities are computed from existing trajectory files and do not provide an independent evaluation of error or control smoothness.

Appendix B. Evaluation metrics and statistical reporting

Table 1 averages four stage rates within each policy–environment unit before summarizing the 15 units. There are five independently trained policies, each with three environment repetitions. Let $c_{u,s}$ and $n_{u,s}$ denote successful and eligible stage-evaluation pairs for unit u and stage s . The following aggregation applies when $n_{u,s} > 0$ for every $u \in \{1, \dots, 15\}$ and $s \in \{1, \dots, 4\}$, so all four stage rates are defined in each unit:

$$p_{u,s} = 100 c_{u,s} / n_{u,s}, \quad m_u = \frac{1}{4} \sum_{s=1}^4 p_{u,s},$$

$$\bar{m} = \frac{1}{15} \sum_{u=1}^{15} m_u, \quad \text{SD}(m) = \sqrt{\frac{1}{14} \sum_{u=1}^{15} (m_u - \bar{m})^2}.$$

The denominator $n_{u,s}$ is the number used in the original stage calculation, not the number of policy seeds or environment seeds. These counts, the task-to-stage assignment, and separate counts for inapplicable, unreached, and missing-frame cases are unavailable. The missing-keyframe exclusion in Table A.2 does not resolve the unreached-stage rule. The formulas specify averaging only under the stated positive-denominator condition. If a unit–stage pair has no eligible evaluations, its rate and the fixed four-stage macro-average are undefined under these formulas. The retained aggregates do not establish whether such cases occurred or how they were handled; no zero imputation, stage exclusion, or weight renormalization is inferred. The

formulas do not reconstruct eligibility or imply that all six tasks contain four complete stages. The macro SD is neither the average of the four stage SDs nor a standard error. Reported table values are retained as aggregates.

Table B.1: Evaluation metrics used in the main text

Metric	Statistical unit	Numerator	Denominator
Stage success (Table 1)	Evaluation episode–stage pair	Successful pairs for the relevant stage	Original eligible stage pairs; counts and unreached-stage rule unavailable
Task success rate	Evaluation episode	Evaluation episodes satisfying the task-success predicate	Held-out evaluation episodes
Collision rate	Complete evaluation episodes; events in Table 7 use their separately specified denominators	Statistical units with environmental or self-collision	Corresponding complete held-out evaluation episodes
Step recovery rate	Trigger segment	Segments satisfying the stage predicate again	All matched compensation-trigger segments
Compensation success rate	Trigger segment	Segments completing the local stage after compensation	All matched trigger segments with defined compensation in Table 4; N/A for Base Policy
Stage failure rate	Evaluation episode–stage pair	Pairs violating the stage predicate	Evaluation pairs for that stage
Trajectory error	Aligned control step	Mean Euclidean deviation from the aligned reference	Aligned samples
Relative bimanual error	Aligned control step	Relative-position deviation between the left and right end effectors	Aligned samples
Smoothness	Three-frame position window	Mean second-order position difference	T minus 2 aligned windows
QP safety correction	Accepted control step	Projection interventions	Per 100 accepted control steps

Table B.2: Random-seed aggregation and matched evaluation protocol

Level	Procedure	Reporting	Interpretation
Policy seed	Train 5 policies independently	Evaluate each policy seed with 3 environment seeds	Identify independent training sources
Environment seed	Evaluate each policy seed under 3 environment seeds	Pair with the same initial-state set, yielding 15 policy–environment evaluation units	Environment repetitions are not independently trained models
Final aggregation	Aggregate the policy–environment evaluation units corresponding to each table	Compute mean and sample standard deviation across corresponding units; compute macro-averages within units first	Interpret dispersion across paired evaluation units as in Section 4.1.3
Matching conditions	Use the same initial states, task protocol, and denominators within each experimental group	Make direct within-group comparisons	Retain distinct statistical populations across tables
Ablations	Switch specified modules according to the component definitions in Table 6; each row follows its listed configuration	Report absolute task, recovery, and collision results	System-configuration comparison; see Section 5 for data and computation boundaries

Appendix C. Supplementary workflow and validity boundaries

This appendix summarizes the end-to-end experimental workflow and the validity boundaries supported by the available evidence.

Table C.1: End-to-end method workflow

Stage	Input	Operation	Output used in the paper
1. Data preparation	RGB/video, robot states, and language; depth used only for proxies and current-scene geometry	Calibration, synchronization, splitting, and normalization	Fixed training, validation, and test sets
2. Base policy	Training demonstrations	Train seed-specific checkpoints using each policy’s recorded configuration; select using validation data only. Budget matching is a design principle; Table A.3 defines what is verified	Structured bimanual VLA and baselines
3. Object reconstruction	Multiview RGB images and object masks	Optimize object-level 3DGS; use depth for metric scene recovery and proxy alignment	Separated visual object assets
4. Physical proxy	Object Gaussians, geometry, and physical priors	Construct collision bodies, movable structures, and Isaac Sim scenes	Interactive digital twins
5. Failure discovery	Current policy and digital twins	Execute the current policy and retain failure states	Failure-region centers
6. Virtual feedback	Structurally feasible candidates	Filter by task, safety, and physics predicates	Accepted trajectories for mixed training
7. Deployment recovery	Current RGB, language, and high-risk local steps	Generate future video, extract and smooth 2-D trajectories, then apply current-state mapping, gating, fusion, and projection	Local actions accepted by shared filtering and nonlinear rechecking, or fallback
8. Evaluation	Held-out evaluation episodes and trigger windows	Use the statistical units and matching protocols corresponding to each table	Tables 1–8: quantitative results; Figs. 6–8 and A.1–A.4: inputs, interactions, and offline diagnostics

Table C.2: Available evidence and applicability boundaries

Issue	Evidence in the paper	Supported conclusion	Remaining boundary
Sparse data	Four stage aggregates; 5 policy seeds	Higher means for the listed configurations	Task counts, stage eligibility, and exclusions unavailable; no quantity-scaling result
Relation supervision	Geometric mapping and combined structured-policy results	Comparison of the structured configuration	Contact/role action mapping and numerical loss settings unavailable; no separate component attribution
Proxy error	Proxy-construction workflow, inspection dimensions, and system-configuration comparisons	System results for configurations involving object-level proxies	Per-object quantitative admission and response errors are not reported
Non-executable video references	Kling AI 3.0, reference-acceptance workflow, and Tables 4–5; offline implementation in Table A.9	References provide rejectable local guidance	Online tracker build, call details, adoption rate, expiration period, and waiting/recovery budgets were not fully retained
Overly conservative safety gating	Recovery, collision, generated-reference rejection, and QP correction statistics	Higher mean recovery and lower mean collision under the current protocol	Filter effects are not isolated; the detection location of the 2.20% joint-structure events is unspecified
Strong-baseline fairness	Within-group data and splits; configuration-to-result mapping in Table A.3	Descriptive system-performance differences among configurations	Backbone checkpoints and complete budgets are not uniquely mapped; GR00T-style is only a configuration reference
Perception and calibration shifts	Lighting, depth, camera-pose, and clutter stress tests	Success and collision rates in the specified perturbation conditions	Perturbation strengths are not listed; no extrapolation to larger shifts
Physics and task shifts	Friction and unseen-object-state stress tests	Performance changes under physics and object-state shifts	Does not establish general recovery outside demonstration neighborhoods

Declaration of generative AI and AI-assisted technologies in manuscript preparation

During manuscript preparation, ChatGPT/Codex (OpenAI) assisted with Chinese-to-English translation, language editing, manuscript organization, reference checking, and LaTeX and diagram preparation. The imagegen tool accessed through Codex was used to explore earlier schematic layouts for Figs. 1–4, which were subsequently rebuilt in PowerPoint. The authors take responsibility for the content of the manuscript. Kling AI 3.0 was used to generate video references as part of the research method, as described in Section 3.3 and Table A.9.

References

- [1] C.-C. Hsu, B. Wen, J. Xu, Y. Narang, X. Wang, Y. Zhu, J. Biswas, S. Birchfield, SPOT: SE(3) Pose Trajectory Diffusion for Object-Centric Manipulation, arXiv preprint arXiv:2411.00965 (2024). doi:10.48550/arXiv.2411.00965.
- [2] J. Kerr, C. M. Kim, M. Wu, B. Yi, Q. Wang, K. Goldberg, A. Kanazawa, Robot See Robot Do: Imitating Articulated Object Manipulation with Monocular 4D Reconstruction, in: Proceedings of The 8th Conference on Robot Learning, Vol. 270 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 587–603.
URL <https://proceedings.mlr.press/v270/kerr25a.html>
- [3] E. Valassakis, G. Papagiannis, N. Di Palo, E. Johns, Demonstrate Once, Imitate Immediately (DOME): Learning Visual Servoing for One-Shot Imitation Learning, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2022, pp. 8614–8621. doi:10.1109/iros47612.2022.9981982.
- [4] M. Xu, Z. Xu, C. Chi, M. Veloso, S. Song, XSkill: Cross Embodiment Skill Discovery, in: Proceedings of The 7th Conference on Robot Learning, Vol. 229 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 3536–3555.
URL <https://proceedings.mlr.press/v229/xu23a.html>
- [5] B. Kerbl, G. Kopanas, T. Leimkuehler, G. Drettakis, 3D Gaussian Splatting for Real-Time Radiance Field Rendering, ACM Transactions on Graphics 42 (4) (2023) 1–14. doi:10.1145/3592433.
- [6] B. Huang, Z. Yu, A. Chen, A. Geiger, S. Gao, 2D Gaussian Splatting for Geometrically Accurate Radiance Fields, in: Special Interest Group on

Computer Graphics and Interactive Techniques Conference Conference Papers, ACM, 2024, pp. 1–11. doi:10.1145/3641519.3657428.

- [7] A. Ajay, S. Han, Y. Du, S. Li, A. Gupta, T. Jaakkola, J. Tenenbaum, L. Kaelbling, A. Srivastava, P. Agrawal, Compositional Foundation Models for Hierarchical Planning, *Advances in Neural Information Processing Systems* 36 (2023) 22304–22325. doi:10.52202/075280-0979.
- [8] H. Bharadhwaj, D. Dwibedi, A. Gupta, S. Tulsiani, C. Doersch, T. Xiao, D. Shah, F. Xia, D. Sadigh, S. Kirmani, Gen2Act: Human Video Generation in Novel Scenarios enables Generalizable Robot Manipulation, *arXiv preprint arXiv:2409.16283* (2024). doi:10.48550/arXiv.2409.16283.
- [9] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, P. Abbeel, Learning Universal Policies via Text-Guided Video Generation, *Advances in Neural Information Processing Systems* 36 (2023) 9156–9172. doi:10.52202/075280-0403.
- [10] J. Liang, R. Liu, E. Ozguroglu, S. Sudhakar, A. Dave, P. Tokmakov, S. Song, C. Vondrick, Dreamitate: Real-World Visuomotor Policy Learning via Video Generation, in: *Proceedings of The 8th Conference on Robot Learning*, Vol. 270 of *Proceedings of Machine Learning Research*, PMLR, 2025, pp. 3943–3960.
URL <https://proceedings.mlr.press/v270/liang25b.html>
- [11] H. Zhen, Q. Sun, H. Zhang, J. Li, S. Zhou, Y. Du, C. Gan, TesserAct: Learning 4D Embodied World Models, *arXiv preprint arXiv:2504.20995* (2025). doi:10.48550/arXiv.2504.20995.
- [12] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, A. Srinivas, Reinforcement Learning with Augmented Data, *Advances in Neural Information Processing Systems* 33 (2020) 19884–19895.
URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/e615c82aba461681ade82da2da38004a-Abstract.html
- [13] D. Yarats, R. Fergus, A. Lazaric, L. Pinto, Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning, *arXiv preprint arXiv:2107.09645* (2021). doi:10.48550/arXiv.2107.09645.
- [14] S. James, Z. Ma, D. R. Arrojo, A. J. Davison, RLBench: The Robot Learning Benchmark & Learning Environment, *IEEE Robotics and Automation Letters* 5 (2) (2020) 3019–3026. doi:10.1109/lra.2020.2974707.
- [15] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: *2017 IEEE/RSJ International Conference on Intelligent*

- Robots and Systems (IROS), IEEE, 2017, pp. 23–30. doi:10.1109/iros.2017.8202133.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning Transferable Visual Models From Natural Language Supervision, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 8748–8763.
URL <https://proceedings.mlr.press/v139/radford21a.html>
 - [17] M. Caron, H. Touvron, I. Misra, H. Jegou, J. Mairal, P. Bojanowski, A. Joulin, Emerging Properties in Self-Supervised Vision Transformers, in: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, 2021, pp. 9630–9640. doi:10.1109/iccv48922.2021.00951.
 - [18] S. Kobayashi, E. Matsumoto, V. Sitzmann, Decomposing NeRF for Editing via Feature Field Distillation, Advances in Neural Information Processing Systems 35 (2022) 23311–23330. doi:10.52202/068431-1694.
 - [19] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, M. Tancik, LERF: Language Embedded Radiance Fields, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, 2023, pp. 19729–19739. doi:10.1109/iccv51070.2023.01807.
 - [20] X. Fu, S. Zhang, T. Chen, Y. Lu, L. Zhu, X. Zhou, A. Geiger, Y. Liao, Panoptic NeRF: 3D-to-2D Label Transfer for Panoptic Urban Scene Segmentation, in: 2022 International Conference on 3D Vision (3DV), IEEE, 2022, pp. 1–11. doi:10.1109/3dv57658.2022.00042.
 - [21] A. Kundu, K. Genova, X. Yin, A. Fathi, C. Pantofaru, L. Guibas, A. Tagliasacchi, F. Dellaert, T. Funkhouser, Panoptic Neural Fields: A Semantic Object-Aware Neural Scene Representation, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2022, pp. 12861–12871. doi:10.1109/cvpr52688.2022.01253.
 - [22] Y. Siddiqui, L. Porzi, S. R. Bulò, N. Müller, M. Nießner, A. Dai, P. Kotschieder, Panoptic Lifting for 3D Scene Understanding With Neural Fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2023, pp. 9043–9052. doi:10.1109/cvpr52729.2023.00873.
 - [23] B. Wang, L. Chen, B. Yang, DM-NeRF: 3D Scene Geometry Decomposition and Manipulation from 2D Images, arXiv preprint arXiv:2208.07227 (2022). doi:10.48550/arXiv.2208.07227.

- [24] D. Rebain, W. Jiang, S. Yazdani, K. Li, K. M. Yi, A. Tagliasacchi, DeRF: Decomposed Radiance Fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2021, pp. 14153–14161. doi:10.1109/cvpr46437.2021.01393.
- [25] V. Tschernezki, I. Laina, D. Larlus, A. Vedaldi, Neural Feature Fusion Fields: 3D Distillation of Self-Supervised 2D Image Representations, in: 2022 International Conference on 3D Vision (3DV), IEEE, 2022, pp. 443–453. doi:10.1109/3dv57658.2022.00056.
- [26] A. Caraffa, D. Boscaini, A. Hamza, F. Poiesi, FreeZe: Training-Free Zero-Shot 6D Pose Estimation with Geometric and Vision Foundation Models, in: European Conference on Computer Vision, Springer, 2024, pp. 414–431. doi:10.1007/978-3-031-73226-3_24.
- [27] Y. He, W. Sun, H. Huang, J. Liu, H. Fan, J. Sun, PVN3D: A Deep Point-Wise 3D Keypoints Voting Network for 6DoF Pose Estimation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11632–11641. doi:10.1109/cvpr42600.2020.01165.
- [28] Y. He, H. Huang, H. Fan, Q. Chen, J. Sun, FFB6D: A Full Flow Bidirectional Fusion Network for 6D Pose Estimation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 3003–3013. doi:10.1109/cvpr46437.2021.00302.
- [29] Y. Labbé, J. Carpentier, M. Aubry, J. Sivic, CosyPose: Consistent Multi-view Multi-object 6D Pose Estimation, in: European Conference on Computer Vision, Springer, 2020, pp. 574–591. doi:10.1007/978-3-030-58520-4_34.
- [30] Y. Labbé, L. Manuelli, A. Mousavian, S. Tyree, S. Birchfield, J. Tremblay, J. Carpentier, M. Aubry, D. Fox, J. Sivic, MegaPose: 6D Pose Estimation of Novel Objects via Render & Compare, in: Proceedings of The 6th Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 715–725.
URL <https://proceedings.mlr.press/v205/labbe23a.html>
- [31] V. N. Nguyen, T. Groueix, M. Salzmann, V. Lepetit, GigaPose: Fast and Robust Novel Object Pose Estimation via One Correspondence, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 9903–9913. doi:10.1109/cvpr52733.2024.00945.
- [32] I. Shugurov, F. Li, B. Busam, S. Ilic, OSOP: A Multi-Stage One Shot Object Pose Estimation Framework, in: Proceedings of the IEEE/CVF Con-

- ference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 6835–6844. doi:10.1109/cvpr52688.2022.00671.
- [33] H. Bharadhwaj, R. Mottaghi, A. Gupta, S. Tulsiani, Track2Act: Predicting Point Tracks from Internet Videos Enables Generalizable Robot Manipulation, in: European Conference on Computer Vision, Springer, 2024, pp. 306–324. doi:10.1007/978-3-031-73116-7_18.
 - [34] P. R. Florence, L. Manuelli, R. Tedrake, Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation, in: Proceedings of The 2nd Conference on Robot Learning, Vol. 87 of Proceedings of Machine Learning Research, PMLR, 2018, pp. 373–385.
URL <https://proceedings.mlr.press/v87/florence18a.html>
 - [35] J. Zhu, Y. Ju, J. Zhang, M. Wang, Z. Yuan, K. Hu, H. Xu, DenseMatcher: Learning 3D Semantic Correspondence for Category-Level Manipulation from a Single Demo, arXiv preprint arXiv:2412.05268 (2024). doi:10.48550/arXiv.2412.05268.
 - [36] L. Barcellona, A. Zadaianchuk, D. Allegro, S. Papa, S. Ghidoni, E. Gavves, Dream to Manipulate: Compositional World Models Empowering Robot Imitation Learning with Imagination, arXiv preprint arXiv:2412.14957 (2024). doi:10.48550/arXiv.2412.14957.
 - [37] A. Correia, L. A. Alexandre, A survey of demonstration learning, Robotics and Autonomous Systems 182 (2024) 104812. doi:10.1016/j.robot.2024.104812.
 - [38] M. Chang, A. Gupta, S. Gupta, Semantic Visual Navigation by Watching YouTube Videos, Advances in Neural Information Processing Systems 33 (2020) 4283–4294.
URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/2cd4e8a2ce081c3d7c32c3cde4312ef7-Abstract.html
 - [39] Y. Liu, W. Dong, Y. Hu, C. Wen, Z.-H. Yin, C. Zhang, Y. Gao, Imitation Learning from Observation with Automatic Discount Scheduling, in: The Twelfth International Conference on Learning Representations, 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/864ab84fabd45a92a93752402db61d23-Abstract-Conference.html
 - [40] D. Patel, H. Eghbalzadeh, N. Kamra, M. L. Iuzzolino, U. Jain, R. Desai, Pretrained Language Models as Visual Planners for Human Assistance, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 15302–15314. doi:10.1109/iccv51070.2023.01404.

- [41] S. Bahl, R. Mendonca, L. Chen, U. Jain, D. Pathak, Affordances From Human Videos as a Versatile Representation for Robotics, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 13778–13790. doi:10.1109/cvpr52729.2023.01324.
- [42] B. Baker, I. Akkaya, P. Zhokov, J. Huizinga, J. Tang, A. Ecoffet, B. Houghton, R. Sampedro, J. Clune, Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos, *Advances in Neural Information Processing Systems* 35 (2022) 24639–24654. doi:10.52202/068431-1789.
- [43] S. Dasari, M. K. Srirama, U. Jain, A. Gupta, An Unbiased Look at Datasets for Visuo-Motor Pre-Training, in: Proceedings of The 7th Conference on Robot Learning, Vol. 229 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 1183–1198.
URL <https://proceedings.mlr.press/v229/dasari23a.html>
- [44] Y. Ju, K. Hu, G. Zhang, G. Zhang, M. Jiang, H. Xu, Robo-ABC: Affordance Generalization Beyond Categories via Semantic Correspondence for Robot Manipulation, in: European Conference on Computer Vision, Springer, 2024, pp. 222–239. doi:10.1007/978-3-031-72940-9_13.
- [45] G. Li, D. Sun, L. Sevilla-Lara, V. Jampani, One-Shot Open Affordance Learning with Foundation Models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 3086–3096. doi:10.1109/cvpr52733.2024.00298.
- [46] G. Li, N. Tsagkas, J. Song, R. Mon-Williams, S. Vijayakumar, K. Shao, L. Sevilla-Lara, Learning Precise Affordances From Egocentric Videos for Robotic Manipulation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025, pp. 10581–10591. doi:10.1109/iccv51701.2025.00985.
- [47] C. Yuan, C. Wen, T. Zhang, Y. Gao, General Flow as Foundation Affordance for Scalable Robot Learning, in: Proceedings of The 8th Conference on Robot Learning, Vol. 270 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 1541–1566.
URL <https://proceedings.mlr.press/v270/yuan25a.html>
- [48] A. G. Billard, S. Calinon, F. Guenter, Discriminative and adaptive imitation in uni-manual and bi-manual tasks, *Robotics and Autonomous Systems* 54 (5) (2006) 370–384. doi:10.1016/j.robot.2006.01.007.
- [49] M. Bombile, A. Billard, Bimanual dynamic grabbing and tossing of objects onto a moving target, *Robotics and Autonomous Systems* 167 (2023) 104481. doi:10.1016/j.robot.2023.104481.

- [50] F. J. Martínez-Peral, J. B. Méndez, D. Mronga, J. V. Segura-Heras, C. Perez-Vidal, Trajectory planning system for bimanual robots: Achieving efficient collision-free manipulation, *Robotics and Autonomous Systems* 194 (2025) 105118. doi:10.1016/j.robot.2025.105118.
- [51] T. Z. Zhao, V. Kumar, S. Levine, C. Finn, Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, in: *Proceedings of Robotics: Science and Systems XIX*, 2023. doi:10.15607/RSS.2023.XIX.016.
- [52] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, C. Finn, OpenVLA: An Open-Source Vision-Language-Action Model, in: *Proceedings of The 8th Conference on Robot Learning*, Vol. 270 of *Proceedings of Machine Learning Research*, PMLR, 2025, pp. 2679–2713.
URL <https://proceedings.mlr.press/v270/kim25c.html>
- [53] J. Cen, Z. Zhou, J. Fang, c. yang, W. Shen, L. Xie, D. Jiang, X. ZHANG, Q. Tian, Segment Anything in 3D with NeRFs, *Advances in Neural Information Processing Systems* 36 (2023) 25971–25990. doi:10.52202/075280-1130.
- [54] M. Oechsle, S. Peng, A. Geiger, UNISURF: Unifying Neural Implicit Surfaces and Radiance Fields for Multi-View Reconstruction, in: *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, 2021, pp. 5569–5579. doi:10.1109/iccv48922.2021.00554.
- [55] J. Luiten, G. Kopanas, B. Leibe, D. Ramanan, Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis, in: *2024 International Conference on 3D Vision (3DV)*, IEEE, 2024, pp. 800–809. doi:10.1109/3dv62453.2024.00044.
- [56] Z. Yang, H. Yang, Z. Pan, L. Zhang, Real-time Photorealistic Dynamic Scene Representation and Rendering with 4D Gaussian Splatting, *arXiv preprint arXiv:2310.10642* (2023). doi:10.48550/arXiv.2310.10642.
- [57] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, X. Jin, Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction, in: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2024, pp. 20331–20341. doi:10.1109/cvpr52733.2024.01922.
- [58] T. Yi, J. Fang, J. Wang, G. Wu, L. Xie, X. Zhang, W. Liu, Q. Tian, X. Wang, GaussianDreamer: Fast Generation from Text to 3D Gaussians by Bridging 2D and 3D Diffusion Models, *arXiv preprint arXiv:2310.08529* (2023). doi:10.48550/arXiv.2310.08529.

- [59] J. Tang, J. Ren, H. Zhou, Z. Liu, G. Zeng, DreamGaussian: Generative Gaussian Splatting for Efficient 3D Content Creation, in: The Twelfth International Conference on Learning Representations, 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/905202e21386913d8eac637c2b50f590-Abstract-Conference.html
- [60] Z. Chen, F. Wang, Y. Wang, H. Liu, Text-to-3D using Gaussian Splatting, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2024, pp. 21401–21412. doi:10.1109/cvpr52733.2024.02022.
- [61] B. Yang, Y. Zhang, Y. Xu, Y. Li, H. Zhou, H. Bao, G. Zhang, Z. Cui, Learning Object-Compositional Neural Radiance Field for Editable Scene Rendering, in: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, 2021, pp. 13759–13768. doi:10.1109/iccv48922.2021.01352.
- [62] C. Wang, M. Chai, M. He, D. Chen, J. Liao, CLIP-NeRF: Text-and-Image Driven Manipulation of Neural Radiance Fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2022, pp. 3835–3844. doi:10.1109/cvpr52688.2022.00381.
- [63] A. Mirzaei, T. Aumentado-Armstrong, K. G. Derpanis, J. Kelly, M. A. Brubaker, I. Gilitschenski, A. Levinstein, SPIn-NeRF: Multiview Segmentation and Perceptual Inpainting With Neural Radiance Fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2023, pp. 20669–20679. doi:10.1109/cvpr52729.2023.01980.
- [64] T. Yu, T. Xiao, J. Tompson, A. Stone, S. Wang, A. Brohan, J. Singh, C. Tan, D. M. J. Peralta, K. Hausman, B. Ichter, F. Xia, Scaling Robot Learning with Semantically Imagined Experience, in: Robotics: Science and Systems XIX, 2023. doi:10.15607/rss.2023.xix.027.
- [65] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, S. Song, Diffusion policy: Visuomotor policy learning via action diffusion, The International Journal of Robotics Research 44 (10–11) (2025) 1684–1704. doi:10.1177/02783649241273668.
- [66] M. Janner, Y. Du, J. B. Tenenbaum, S. Levine, Planning with Diffusion for Flexible Behavior Synthesis, arXiv preprint arXiv:2205.09991 (2022). doi:10.48550/arXiv.2205.09991.
- [67] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap, Mastering Diverse Domains through World Models, arXiv preprint arXiv:2301.04104 (2023). doi:10.48550/arXiv.2301.04104.

- [68] G. Chalvatzaki, A. Younes, D. Nandha, A. T. Le, L. F. R. Ribeiro, I. Gurevych, Learning to reason over scene graphs: a case study of fine-tuning GPT-2 into a robot language model for grounded task planning, *Frontiers in Robotics and AI* 10 (2023) 1221739. doi:10.3389/frobt.2023.1221739.
- [69] Z.-H. Yin, S. Yang, P. Abbeel, Object-centric 3D Motion Field for Robot Learning from Human Videos, arXiv preprint arXiv:2506.04227 (2025). doi:10.48550/arXiv.2506.04227.
- [70] Z. Dong, K. Chen, Z. Lv, H.-X. Yu, Y. Zhang, C. Zhang, Y. Zhu, S. Tian, Z. Li, G. Moffatt, S. Christofferson, J. Fort, X. Pan, M. Yan, J. Wu, C. Y. Ren, R. Newcombe, Digital Twin Catalog: A Large-Scale Photorealistic 3D Object Digital Twin Dataset, in: 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2025, pp. 753–763. doi:10.1109/cvpr52734.2025.00079.
- [71] A. Capitanelli, M. Maratea, F. Mastrogiovanni, M. Vallati, On the manipulation of articulated objects in human–robot cooperation scenarios, *Robotics and Autonomous Systems* 109 (2018) 139–155. doi:10.1016/j.robot.2018.08.003.
- [72] X. Li, J. Li, Z. Zhang, R. Zhang, F. Jia, T. Wang, H. Fan, K.-K. Tseng, R. Wang, RoboGSim: A Real2Sim2Real Robotic Gaussian Splatting Simulator, arXiv preprint arXiv:2411.11839 (2024). doi:10.48550/arXiv.2411.11839.
- [73] Y. Mu, T. Chen, Z. Chen, S. Peng, Z. Lan, Z. Gao, Z. Liang, Q. Yu, Y. Zou, M. Xu, L. Lin, Z. Xie, M. Ding, P. Luo, RoboTwin: Dual-Arm Robot Benchmark with Generative Digital Twins, arXiv preprint arXiv:2504.13059 (2025). doi:10.48550/arXiv.2504.13059.
- [74] J. Yu, L. Fu, H. Huang, K. El-Refai, R. A. Ambrus, R. Cheng, M. Z. Irshad, K. Goldberg, Real2Render2Real: Scaling Robot Data Without Dynamics Simulation or Robot Hardware, in: Proceedings of The 9th Conference on Robot Learning, Vol. 305 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 547–577.
URL <https://proceedings.mlr.press/v305/yu25a.html>
- [75] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, D. Fox, MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations, in: Proceedings of The 7th Conference on Robot Learning, Vol. 229 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 1820–1864.
URL <https://proceedings.mlr.press/v229/mandlekar23a.html>

- [76] M. Albaba, C. Li, M. Diomataris, O. Taheri, A. Krause, M. Black, NIL: No-data Imitation Learning by Leveraging Pre-trained Video Diffusion Models, arXiv preprint arXiv:2503.10626 (2025). doi:10.48550/arXiv.2503.10626.
- [77] X. He, J. Sun, Y. Wang, D. Huang, H. Bao, X. Zhou, OnePose++: Keypoint-Free One-Shot Object Pose Estimation without CAD Models, Advances in Neural Information Processing Systems 35 (2022) 35103–35115. doi:10.52202/068431-2544.
- [78] Y. He, Y. Wang, H. Fan, J. Sun, Q. Chen, FS6D: Few-Shot 6D Pose Estimation of Novel Objects, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 6814–6824. doi:10.1109/cvpr52688.2022.00669.
- [79] F. Li, S. R. Vutukur, H. Yu, I. Shugurov, B. Busam, S. Yang, S. Ilic, NeRF-Pose: A First-Reconstruct-Then-Regress Approach for Weakly-Supervised 6D Object Pose Estimation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops, 2023, pp. 2123–2133. doi:10.1109/iccvw60793.2023.00226.
- [80] Y. Liu, Y. Wen, S. Peng, C. Lin, X. Long, T. Komura, W. Wang, Gen6D: Generalizable Model-Free 6-DoF Object Pose Estimation from RGB Images, in: European Conference on Computer Vision, Springer, 2022, pp. 298–315. doi:10.1007/978-3-031-19824-3_18.
- [81] J. Sun, Z. Wang, S. Zhang, X. He, H. Zhao, G. Zhang, X. Zhou, OnePose: One-Shot Object Pose Estimation Without CAD Models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 6825–6834. doi:10.1109/cvpr52688.2022.00670.
- [82] V. Lepetit, F. Moreno-Noguer, P. Fua, EPnP: An Accurate O(n) Solution to the PnP Problem, International Journal of Computer Vision 81 (2) (2009) 155–166. doi:10.1007/s11263-008-0152-6.
- [83] K. Park, T. Patten, M. Vincze, Pix2Pose: Pixel-Wise Coordinate Regression of Objects for 6D Pose Estimation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 7668–7677. doi:10.1109/iccv.2019.00776.
- [84] J. Tremblay, T. To, B. Sundaralingam, Y. Xiang, D. Fox, S. Birchfield, Deep Object Pose Estimation for Semantic Robotic Grasping of Household Objects, in: Proceedings of The 2nd Conference on Robot Learning, Vol. 87 of Proceedings of Machine Learning Research, PMLR, 2018, pp. 306–316. URL <https://proceedings.mlr.press/v87/tremblay18a.html>
- [85] J. Wasserman, K. Yadav, G. Chowdhary, A. Gupta, U. Jain, Last-Mile

- Embodied Visual Navigation, in: Proceedings of The 6th Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 666–678.
URL <https://proceedings.mlr.press/v205/wasserman23a.html>
- [86] J. Wasserman, G. Chowdhary, A. Gupta, U. Jain, Exploitation-Guided Exploration for Semantic Embodied Navigation, in: 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2024, pp. 2901–2908. doi:10.1109/icra57147.2024.10610117.
 - [87] P. Ausserlechner, D. Habegger, S. Thalhammer, J.-B. Weibel, M. Vincze, ZS6D: Zero-shot 6D Object Pose Estimation using Vision Transformers, in: 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2024, pp. 463–469. doi:10.1109/icra57147.2024.10611464.
 - [88] B. Wen, W. Yang, J. Kautz, S. Birchfield, FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 17868–17879. doi:10.1109/cvpr52733.2024.01692.
 - [89] S. Calinon, A tutorial on task-parameterized movement learning and retrieval, *Intelligent Service Robotics* 9 (1) (2016) 1–29. doi:10.1007/s11370-015-0187-9.
 - [90] Z. Luo, J. Cao, A. Winkler, K. Kitani, W. Xu, Perpetual Humanoid Control for Real-time Simulated Avatars, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 10895–10904. doi:10.1109/iccv51070.2023.01000.
 - [91] S. Niekum, S. Osentoski, G. Konidaris, A. G. Barto, Learning and generalization of complex tasks from unstructured demonstrations, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2012, pp. 5239–5246. doi:10.1109/iros.2012.6386006.
 - [92] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, A. Kanazawa, AMP: adversarial motion priors for stylized physics-based character control, *ACM Transactions on Graphics* 40 (4) (2021) 1–20. doi:10.1145/3450626.3459670.
 - [93] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, X. Wang, Expressive Whole-Body Control for Humanoid Robots, arXiv preprint arXiv:2402.16796 (2024). doi:10.48550/arXiv.2402.16796.
 - [94] S. Choi, M. Pan, J. Kim, Nonparametric Motion Retargeting for Humanoid Robots on Shared Latent Space, in: *Robotics: Science and Systems*, 2020. doi:10.15607/rss.2020.xvi.071.
 - [95] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, G. Shi, Learning

- Human-to-Humanoid Real-Time Whole-Body Teleoperation, in: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2024, pp. 8944–8951. doi:10.1109/iros58592.2024.10801984.
- [96] D. Kulić, E. A. Croft, Real-time safety for human–robot interaction, *Robotics and Autonomous Systems* 54 (1) (2006) 1–12. doi:10.1016/j.robot.2005.10.005.